

shmap

Sketch-Based Long-Read Mapping
Algorithm and Implementation Reference

From FracMinHash Sketching to Bucketed Seed-and-Extend Mapping

Technical reference compiled from source analysis and a full independent re-implementation

July 19, 2026

Abstract

This document is a complete algorithmic and implementation reference for **shmap** (also known as *Map-SHmap* / *sweepmap*), a sketch-based long-read mapper. It is written to be sufficient, on its own, to re-implement the tool from scratch in any language: every data structure, every formula, every loop invariant, and every edge case is specified precisely, with excerpts from both the original C++ implementation and an independent Rust re-implementation produced during the analysis that underlies this document. The exercise of producing a line-for-line port surfaced several defects and dead code paths in the reference implementation; these are documented explicitly (in “Known defect” boxes) so that a from-scratch implementer can make an informed choice about whether to reproduce or correct them.

The document begins with the theoretical background required to understand the algorithm (k-mers, canonical strands, minimizer-style sketching, FracMinHash / Mash-style sketching, Jaccard and containment similarity, seed-and-extend mapping, longest increasing subsequence chaining, and mapping-quality estimation), then describes the system architecture, then proceeds module-by-module through the full implementation, and closes with complexity analysis, a full end-to-end pseudocode listing, a CLI/parameter reference, and an appendix cataloguing every discovered defect.

Contents

1	Introduction	5
1.1	The long-read mapping problem	5
1.2	shmap’s approach in one paragraph	5
1.3	How to use this document	6
2	Theoretical Background	7
2.1	k -mers	7
2.1.1	Canonical k -mers and strand	7
2.2	Hashing k -mers, and rolling hashes	8
2.3	Sketching: reducing k -mer sets to a manageable, comparable size	8
2.3.1	Minimizer schemes	8
2.3.2	MinHash	8
2.3.3	FracMinHash (“Mash Screen” / scaled MinHash)	9
2.3.4	FracMinHash vs. minimizers, concretely	9
2.4	Seed-and-extend / seed-chain-extend mapping	10
2.5	Co-linear chaining via Longest Increasing / Common Subsequence	10
2.6	The seed heuristic: a provable upper bound for cheap pruning	10
2.7	Mapping quality (MAPQ)	11
2.8	Summary of the theoretical pipeline	12
3	General Architecture	13
3.1	Two phases: indexing and mapping	13
3.2	Configuration axes: the three compile-time flags	13
3.3	The metric axis	15
3.4	Data-flow summary: what crosses each module boundary	16
3.5	Complexity at a glance	16
4	Core Data Types	17
4.1	Scalar type aliases	17
4.2	<code>Kmer</code>	17
4.3	<code>Hit</code>	18
4.4	<code>Seed</code>	18
4.5	<code>Match</code>	19
4.6	<code>BucketLoc</code>	19
4.7	<code>BucketContent</code>	19
4.8	<code>RefSegment</code>	20
4.9	Hash-map type aliases	20
4.10	Mapping and its stats sub-structures	21

5	FracMinHash Sketching	23
5.1	Inputs and outputs	23
5.2	Lookup tables: per-nucleotide hash contributions	23
5.3	The position semantics of <code>Kmer.r</code>	24
5.4	The rolling-hash recurrence	24
5.5	Reference C++ implementation	25
5.6	Rust re-implementation	26
5.7	Non-ACGT input	27
5.8	Complexity	27
5.9	Worked example	27
6	Reference Indexing	28
6.1	Data owned by the index	28
6.2	Building the index: <code>build_index</code>	28
6.2.1	<code>populate_h2pos</code> : routing each hit to the right map	29
6.2.2	Sorting multi-hit lists	29
6.2.3	<code>erase_frequent_kmers</code> : capping pathologically repetitive k -mers	29
6.3	<code>count(h)</code> : reference occurrence count for a hash	30
6.4	Reference C++ implementation	30
6.5	Complexity and memory	30
7	Seeding	32
7.1	Grouping: <code>unique_elements_with_info</code>	32
7.1.1	Reference C++ implementation	33
7.2	The seed budget S	34
7.3	<code>more_seeds_if_cheap</code> : opportunistic budget extension (dead code)	34
8	Bucketing	35
8.1	Bucket geometry	35
8.1.1	Choosing ℓ : the bucket half-length	36
8.2	Buckets: the primary per-run accumulator	36
8.2.1	<code>clear()</code>	38
8.2.2	<code>add_to_pos</code> and <code>add_to_bucket</code>	38
8.2.3	Propagating the seed cursor: <code>propagate_seeds_to_buckets</code>	38
8.2.4	<code>get_sorted_buckets</code>	39
8.3	BucketsHash: ephemeral scratch structure for one multi-hit seed	39
8.4	<code>match_seeds</code> : populating buckets from the seed budget	39
8.5	Reference C++ implementation	40
8.6	Complexity	41
9	Seed-Heuristic Pruning	42
9.1	<code>hseed</code> : the bound formula	42
9.2	<code>matches_in_bucket</code> : extending a bucket by one more seed	42
9.3	<code>seed_heuristic_pass</code> : the driving loop	43
9.4	The <code>no_bucket_pruning</code> flag (-P)	44
9.5	Reference C++ implementation	44
9.6	Worked example	45

10 Refinement and Scoring	46
10.1 The <code>diff_hist</code> bookkeeping device	46
10.2 SHMapper's own <code>bestFixedLength</code> : the main refinement sweep	47
10.3 <code>findBestMapping</code> : dispatching on metric	48
10.4 The <code>bucket_LCS</code> metric: longest common subsequence chain score	49
10.4.1 <code>lcs_get_ppos_in_t</code> : collecting the position sequence	49
10.4.2 <code>lcs_get_lis</code> : patience-sorting LIS length	49
10.4.3 <code>lcs</code> : combining forward and reverse	50
10.5 <code>match_rest</code> : best and second-best search	50
10.5.1 Overlap exclusion	50
10.5.2 How the caller invokes <code>match_rest</code> twice	51
10.6 The ground-truth-only sibling functions, and a genuine bug	51
10.7 Complexity	51
11 End-to-End Algorithm	52
11.1 <code>map_reads</code> : the outer loop	52
11.2 <code>map_read</code> : per-read driver	53
11.2.1 Counter initialization	53
11.2.2 Output: mapped vs. unmapped	54
11.3 Full call graph	55
11.4 Reference C++ implementation (annotated)	55
12 Mapping Quality Estimation	57
12.1 Inputs	57
12.2 <code>set_second_best</code>	57
12.3 <code>calc_mapq</code> : the concrete formula	57
12.4 Two commented-out alternative formulas	58
12.5 Reference C++ implementation	59
12.6 When is <code>calc_mapq</code> actually invoked	59
13 Output Format	60
13.1 The 12 mandatory PAF columns	60
13.2 Custom diagnostic tags (mapped reads)	61
13.2.1 Per-candidate tags (<code>LocalMappingStats</code>)	62
13.2.2 Run-wide/per-read summary tags (<code>GlobalMappingStats</code>)	63
13.3 Ground-truth diagnostic tags (<code>-v 2</code> only)	63
13.4 Unmapped-read record	64
13.5 Run-wide summary printed to <code>stderr</code>	64
14 Command-Line Parameter Reference	66
14.1 Parameters not exposed on the CLI but referenced throughout this document	68
15 Complexity Analysis	69
15.1 Indexing (once per run)	69
15.2 Per read	70
15.3 Memory	71
15.4 Practical scaling knobs	72
16 Catalog of Defects and Dead Code in the Reference Implementation	73
16.1 Summary table	74
16.2 D1: Uninitialized hash lookup tables	75
16.3 D2: <code>kmers_notmatched</code> always reports the full sketch size	75
16.4 D3: <code>more_seeds_if_cheap</code> is fully implemented but unreachable	75

16.5	D4: Hardcoded 100-segment cap	75
16.6	D5–D7: Dead stubs and unreachable methods	75
16.7	D8: Dead <code>lmax</code> parameter in the ground-truth <code>bestFixedLength</code>	76
16.8	D9: Off-by-one in <code>bestIncludedJaccard</code> (real bug)	76
16.9	D10: Missing bounds clamp in ground-truth <code>collect_matches</code>	76
16.10	D11: <code>RefSegment::seq</code> is stored but never read (in the shipped feature set)	76
16.11	D12: Dead <code>-a</code> /SAM output path	77
16.12	D13: Dead <code>-S/-n</code> CLI flags	77
16.13	D14: <code>one_sweep/-B</code> has no implemented behavior	77
16.14	D15: Per-read counters are never actually reset (real bug, affects live output)	77
16.15	D16: Unconditional dereference of an empty optional on unmapped reads (real bug, UB)	78
16.16	D17: <code>gt_c_r_lmax</code> is declared, printed, but never assigned	78
16.17	D18: PAF columns 10/11 always repeat the query-length column	78
16.18	D19: <code>Timers::secs</code> 's exception handling can never actually fire	79
16.19	D20: <code>ParsedQueryId::parse</code> catches a parse error only to rethrow it	79
A	Notation, Glossary, and Quick Index	80
A.1	Notation summary	80
A.2	Glossary	80
A.3	Quick algorithm index	81
A.4	Minimal reference implementation checklist	81

Chapter 1

Introduction

1.1 The long-read mapping problem

Given a *reference* sequence (or a set of reference segments, e.g. chromosomes/contigs) $T = T_1, \dots, T_s$ over the alphabet $\Sigma = \{A, C, G, T\}$, and a *query* read P (typically 1–100+ kbp for long-read sequencing technologies such as PacBio HiFi or Oxford Nanopore), the mapping problem is: find the location(s) $(i, [l, r))$ — segment index i and half-open interval $[l, r) \subseteq [0, |T_i|)$ — such that $T_i[l..r)$ is the region of the reference that P most likely originated from, together with a confidence score (mapping quality, MAPQ).

This differs from *alignment*: mapping only needs to find the correct region and strand, not compute a base-level edit script (CIGAR). Mapping is the first, and usually the computational bottleneck, phase of most long-read tools (minimap2, Winnowmap, MashMap, mapquik, and shmap all separate mapping from an optional downstream alignment phase).

The naive solution — align P against every possible substring of T with a full dynamic program (edit distance / Smith–Waterman) — costs $O(|T| \cdot |P|)$ per read and is many orders of magnitude too slow for genome-scale references ($|T| \sim 10^9$ – 10^{10}) and population-scale read sets (10^5 – 10^7 reads). All practical long-read mappers instead use a **seed-and-extend** (or *seed-chain-extend*) strategy: cheaply find a small number of short, exact-matching anchors (*seeds*) between P and T , then use only the anchors’ positions — not the sequence content — to decide where P came from.

1.2 shmap’s approach in one paragraph

shmap represents both the reference and each read as a **FracMinHash sketch**: a size-proportional, hash-thresholded subset of the sequence’s k -mers (Chapter 2 develops this precisely). Reference k -mers are indexed once into a hash table mapping k -mer hash $\rightarrow$ list of reference positions. For each read, its own sketch is grouped into *seeds* (rarest k -mers first) and matched against the index; matches are accumulated into overlapping, fixed-length *buckets* along each reference segment. A cheap, provably-safe *seed heuristic* upper bound is used to prune buckets that cannot possibly beat the current best candidate before any expensive refinement is attempted on them. Buckets that survive pruning are refined with an exact two-pointer / sliding-window sweep (or, in alternate configurations, a longest-common-subsequence chain) to produce a Jaccard or containment similarity score; the best and second-best surviving mappings are used to compute a mapping-quality estimate; and the result is emitted in PAF format with a rich set of custom diagnostic tags.

Two design choices set shmap apart from more familiar tools like minimap2:

1. **FracMinHash sketching instead of minimizer windows.** Rather than picking one minimal-hash k -mer per sliding window (a *minimizer* scheme), shmap keeps every k -mer

whose hash falls below a fixed fraction of the hash space. This makes sketch size *proportional to sequence length* with a tunable ratio r (flag `-r`), rather than proportional to (length / window size); see §2.3.4.

2. **Score-bound-driven bucket pruning instead of chaining every anchor.** Instead of running a full co-linear chaining dynamic program over every seed hit (as minimap2 does), shmap only ever considers small, fixed-geometry *buckets* of the reference, and prunes the overwhelming majority of them using a cheap closed-form upper bound on the score they could possibly achieve, before doing any real work on them. This is the “SH” (seed heuristic) in the tool’s Map-SHmap name.

1.3 How to use this document

The chapters are ordered so that a reader with no prior background in sketch-based mapping can start at Chapter 2 and, by the time they reach the per-module chapters (Chapters 5–10), already understand *why* every formula and data structure exists, not just *what* it computes. Each per-module chapter gives:

- the exact input/output contract of every function (types, units, invariants assumed and guaranteed);
- a boxed pseudocode listing that is language-independent and directly implementable;
- excerpts of the original C++ (from the `shmap` repository, files under `src/`) and of an independent Rust re-implementation, side by side, wherever the exact bit-level or control-flow behavior matters;
- worked numeric examples for the trickiest arithmetic (bucket geometry, the FracMinHash rolling hash, the seed-heuristic bound);
- boxed notes on defects discovered by cross-checking the two implementations against each other and against the algorithm’s own stated invariants — several of which are not merely cosmetic (Chapter 16 collects all of them with full detail).

Chapter 11 then reassembles every module into one top-to-bottom pseudocode listing of the per-read mapping routine, which is the single most useful reference for a from-scratch implementation. Chapter A collects notation, a symbol table, and a condensed algorithm index for quick lookup while implementing.

Chapter 2

Theoretical Background

This chapter develops, from first principles, every theoretical tool shmap relies on. No prior familiarity with sketch-based mapping is assumed.

2.1 k -mers

A **k -mer** is a contiguous substring of length k of a DNA sequence. A sequence S of length n over $\Sigma = \{A, C, G, T\}$ has exactly $n - k + 1$ k -mers (for $n \geq k$), one starting at each position $i \in [0, n - k]$: $S[i \dots i+k)$.

k -mers are the atomic unit of comparison in almost all modern sequence analysis: instead of comparing sequences character-by-character (which is what alignment does, and is expensive), tools compare the *sets* of k -mers two sequences share. Two sequences that share many k -mers are likely to be similar/overlapping/from the same genomic locus; two that share almost none are likely unrelated. This is the basis of alignment-free sequence comparison.

Two design parameters govern how well k -mers work as a similarity proxy:

- If k is too small, k -mers recur very frequently by chance alone (there are only 4^k distinct k -mers; for $k = 8$ that is 65,536, tiny compared to a genome of 10^9 bases), so a shared k -mer carries little evidence of true homology.
- If k is too large, a single sequencing error (common in long reads, particularly Oxford Nanopore data, at rates of 1–15% depending on chemistry) destroys every k -mer that overlaps it (up to k of them), so real matches are missed.

shmap’s default is $k = 15$ (flag `-k`), with typical experiments in the codebase’s own evaluation harness sweeping $k \in \{17, 19, \dots, 33\}$.

2.1.1 Canonical k -mers and strand

DNA is double-stranded, and a sequencing read may originate from either strand, with no way to know which *a priori*. A k -mer x occurring on the forward strand of the reference is identical to the **reverse complement** of x occurring on the reverse strand at the mirrored position. To make k -mer matching strand-agnostic, essentially every tool in this space computes, for each k -mer window, both:

- the forward-strand hash h_{fw} , and
- the reverse-complement hash h_{rc} (i.e., the hash of the reverse complement of the same window),

and uses a canonical combination of the two (commonly $\min(h_{fw}, h_{rc})$, or, as shmap does, $h_{fw} \oplus h_{rc}$ — see §5.4) as the k -mer’s identity, plus a single bit recording which strand “won” so that downstream code can recover orientation.

2.2 Hashing k -mers, and rolling hashes

Because a sequence of length n has $O(n)$ overlapping k -mers, naively hashing each one from scratch costs $O(nk)$. A **rolling hash** instead updates the previous window’s hash in $O(1)$ to obtain the next window’s hash, giving $O(n)$ total. The general recurrence for a polynomial or XOR-based rolling hash removes the outgoing base’s contribution and adds the incoming base’s contribution:

$$h_{i+1} = f(h_i, \text{out_base} = S[i], \text{in_base} = S[i + k])$$

shmap uses an XOR/rotate rolling hash (not a polynomial one); the exact recurrence, including the reverse-complement side, is derived in §5.4.

2.3 Sketching: reducing k -mer sets to a manageable, comparable size

Even with an efficient rolling hash, a set of $O(n)$ k -mers per sequence is too large to compare pairwise across, e.g., every read against a whole genome. **Sketching** techniques subsample the k -mer set down to a small representative subset, in a way that lets similarity between two sequences be *estimated* from the subsets alone, with a distortion (approximation error) that shrinks as the subset grows.

2.3.1 Minimizer schemes

The most widely used sketching scheme (used by minimap2, most short-read aligners’ seed indices, Winnowmap, etc.) is the **minimizer**: slide a window of w consecutive k -mers along the sequence, and keep only the k -mer with the smallest hash value in each window (ties broken by position or a secondary key). This guarantees:

- **Bounded density**: at most $2/(w + 1)$ of all k -mers are ever selected (in expectation, for a uniform hash), giving a sketch size proportional to n/w .
- **Window guarantee**: any window of w consecutive k -mers has *at least one* selected minimizer, so no region of the sequence is left completely unsampled — important for finding short matches / handling indels robustly.

The window guarantee is minimizers’ main strength; their main weakness is that the selected set is sensitive to single-base edits near a window boundary (a phenomenon studied extensively in the minimizer/syncmer literature), and that sketch size is governed by w , not by a similarity-space quantity directly.

2.3.2 MinHash

An older, different sketching idea (Broder, 1997) underlies Mash and sourmash. Given a hash function h assumed to distribute k -mers uniformly over $[0, H_{\max}]$, the **MinHash sketch** of a k -mer set A of size n keeps only the n smallest-hash elements for a fixed target sketch size (*bottom- n sketch*), or equivalently the single minimum hash value repeated over many independent hash functions (*classic MinHash*). The key theoretical result is:

$$\Pr[\min h(A) = \min h(B)] = \frac{|A \cap B|}{|A \cup B|} = J(A, B)$$

i.e., the probability that two sets’ minimum hashes coincide equals their **Jaccard similarity**. Averaging this indicator over m independent hash functions (or, equivalently, comparing the m smallest of a single hash function’s outputs) gives an unbiased estimator of $J(A, B)$ with standard error $O(1/\sqrt{m})$.

MinHash’s practical drawback for genomic sketching: a fixed-size bottom- n sketch of a short sequence and of a long sequence look the same size, so a short sequence’s sketch is a much denser (and hence unrepresentative, if n is not scaled per-sequence) sample of its k -mers. This motivates **FracMinHash**.

2.3.3 FracMinHash (“Mash Screen” / scaled MinHash)

FracMinHash (used by Mash Screen, sourmash’s “scaled” sketches, and shmap) fixes not a sketch *size* but a hash-space **fraction** $r \in (0, 1]$ (shmap’s flag `-r`, default 0.05): a k -mer with hash h is kept in the sketch iff

$$h \leq r \cdot H_{\max}$$

where $H_{\max}$ is the maximum possible hash value (for a 64-bit hash, $H_{\max} = 2^{64} - 1$). Because h is (assumed) uniformly distributed, this keeps, in expectation, a fraction r of all k -mers, so **sketch size scales linearly with sequence length**: $|\text{sketch}(S)| \approx r \cdot (|S| - k + 1)$, for any $|S|$. This is the key advantage over both fixed-size MinHash and (to a lesser extent) minimizers: one parameter r controls sampling density uniformly across sequences of any length, and, unlike bottom- n MinHash, two FracMinHash sketches computed with the *same* threshold are directly comparable set-theoretically (their union/intersection correspond exactly to the union/intersection of the full k -mer sets restricted to the sub-threshold region) — no renormalization needed.

Jaccard and containment estimation. For two sketches $\hat{A} = A \cap [0, rH_{\max}]$ and $\hat{B} = B \cap [0, rH_{\max}]$ built with the *same* threshold,

$$\widehat{J(A, B)} = \frac{|\hat{A} \cap \hat{B}|}{|\hat{A} \cup \hat{B}|} \quad \text{estimates} \quad J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

and, when $|A| \ll |B|$ (e.g. A a read, B the whole genome) the more relevant **containment index**

$$C(A, B) = \frac{|A \cap B|}{|A|}$$

(“what fraction of the read’s k -mers are found somewhere in the reference”) is estimated directly (without needing to also estimate $|A \cup B|$) as

$$\widehat{C(A, B)} = \frac{|\hat{A} \cap \hat{B}|}{|\hat{A}|}.$$

shmap’s two headline metrics, **Containment** and **Jaccard** (CLI flag `-m`), are exactly these two estimators, computed not over the whole reference but over a small candidate *bucket* region (Chapter 10); shmap additionally offers two alternative, bucket-native metrics (`bucket_SH`, `bucket_LCS`) discussed in §??.

2.3.4 FracMinHash vs. minimizers, concretely

		Minimizer (w -window)	FracMinHash (ratio r)
Sketch size		$\approx 2n/(w+1)$	$\approx rn$
Coverage guarantee		every w -window has ≥ 1 selected k -mer	none: a long low-hash-density stretch can (rarely) be unsampled
Comparable lengths	across	only if w fixed and sequences aligned to the same coordinate frame	yes, directly, by construction
Used by		minimap2, Winnowmap, most short-read aligners	Mash Screen, sourmash, shmap

shmap explicitly favors the containment/Jaccard-estimation cleanliness of FracMinHash over minimizers’ coverage guarantee; its seed-heuristic pruning (§2.6) is in fact a direct consequence of working in FracMinHash’s clean estimation framework.

2.4 Seed-and-extend / seed-chain-extend mapping

Given sketches of P (the read) and T (the reference), the generic mapping pipeline is:

1. **Seed:** find every position where a k -mer of P ’s sketch also occurs in T ’s sketch (a *hit*); a hit is a pair (position in P , position in T).
2. **Chain / cluster:** group hits that are roughly co-linear (i.e., consistent with one contiguous alignment: as position-in- P increases, position-in- T increases by a comparable amount, same strand) into candidate regions.
3. **Score / extend:** for each candidate region, compute a similarity score (or, in tools that go further, a base-level alignment) and report the best (and, for mapping quality, second-best) candidate.

Tools differ enormously in step 2. minimap2 runs an $O(N \log N)$ or $O(N^2)$ (bounded) dynamic program over all N hits to find the highest-scoring chain of co-linear anchors (*co-linear chaining*). shmap instead partitions the reference into fixed-geometry, overlapping **buckets** (a form of coarse, position-only clustering, described in Chapter 8) and scores whole buckets rather than chaining individual hits — far cheaper, at the cost of being less precise about the exact alignment boundary (which shmap does not need, since it is a mapper, not an aligner).

2.5 Co-linear chaining via Longest Increasing / Common Subsequence

One of shmap’s four scoring metrics, `bucket_LCS` (§??), scores a candidate bucket by the length of the **longest common subsequence** (LCS) between the order hits occur in P and the order they occur in T within the bucket. Because both sequences here are permutations of (a subset of) the same multiset of positions-in- P , “LCS” reduces to **longest increasing subsequence** (LIS) of the sequence of P -positions visited in increasing T -order: a long increasing run means many hits are mutually co-linear (consistent with one real alignment on one strand); a long *decreasing* run means many hits are co-linear on the *other* strand. shmap computes both (forward and reverse) and takes the max (§10.4).

Patience-sorting LIS algorithm. The classic $O(n \log n)$ algorithm for LIS length (not full reconstruction) processes the sequence left to right, maintaining an array *tails* where *tails*[i] is the smallest possible tail value of any increasing subsequence of length $i+1$ seen so far. For each new value x : binary-search *tails* for the first entry $\geq x$; if none exists, append x (extends the longest run found so far by one); otherwise overwrite that entry with x (keeps the same achievable lengths but with a smaller, more extensible tail). The final array length is the LIS length. This is exactly the algorithm shmap’s `lcs.get_lis` implements (§10.4).

2.6 The seed heuristic: a provable upper bound for cheap pruning

This is the theoretical core that gives Map-SHmap its name and its main performance advantage over exhaustively scoring every candidate region.

Consider a read sketch of size $m = |\hat{P}|$ (its FracMinHash sketch cardinality). Seeds are consumed *rarest-first* (Chapter 7) up to some budget $S \leq m$ seeds. For any candidate bucket, define:

- *seeds* = number of seed *occurrences* (counting multiplicity in P) that have been consumed and attributed to this bucket so far (an occurrence is attributed to a bucket the moment its position is examined — whether or not the reference actually has a hit there);
- *matches* = number of those occurrences that *did* find a hit landing inside this bucket.

Necessarily $matches \leq seeds \leq m$ (a seed occurrence can match at most once per bucket, and only $seeds \leq m$ occurrences from the read have been consumed at all). Define the **seed-heuristic score**

$$h_{\text{seed}}(m, seeds, matches) = 1 - \frac{seeds - matches}{m}.$$

Claim. h_{seed} is a non-increasing function of *seeds* for fixed *matches*, and an upper bound on the true containment score $C = matches_{\text{final}}/m$ that this bucket could achieve *even if every remaining, not-yet-consumed seed occurrence in P also matched inside this bucket*.

Why. At most m total seed occurrences exist; $seeds - matches$ of the ones examined so far are *known misses* for this bucket (an occurrence was consumed and did *not* land in this bucket). Even in the best possible case — every one of the $m - seeds$ not-yet-examined occurrences turns out to match this bucket too — the final match count can be at most $m - (seeds - matches)$ (total occurrences minus known misses), giving final containment score at most $(m - (seeds - matches))/m = h_{\text{seed}}$. Since known misses only ever accumulate (both *seeds* and *seeds - matches* are non-decreasing as more occurrences are examined), h_{seed} can only decrease or stay the same as more seeds are consumed for a fixed bucket — it is a monotonically tightening upper bound.

Pruning rule. If, after extending a bucket with more seed occurrences, $h_{\text{seed}} < \theta$ for the current acceptance threshold θ (or the current best-known score, whichever is the active bound), the bucket is *guaranteed* to be unable to reach θ no matter how the remaining seeds resolve, and can be discarded *without computing its actual score*. This is what makes shmap fast: the overwhelming majority of candidate buckets a repetitive genome generates are eliminated by this $O(1)$ -per-check bound long before the $O(\text{bucket size})$ refinement sweep (Chapter 10) would ever run on them. Chapter 9 gives the exact incremental algorithm (`seed_heuristic_pass`) that extends a bucket with just enough additional seeds to either prove it’s prunable or exhaust all seeds, at the smallest possible cost.

2.7 Mapping quality (MAPQ)

A mapper is rarely certain a single location is correct, especially in repetitive genomic regions where a read may score similarly against several loci. **MAPQ** (following the SAM/PAF convention, an integer typically in $[0, 60]$, though PAF’s own histrionic convention allows 255 for “unavailable”) encodes, on a Phred-like scale, the mapper’s confidence that the reported best location is correct. The general recipe used by essentially all mappers (originating with MAQ/BWA, and reused with variations by minimap2 and shmap) is:

1. Find the best-scoring candidate location, score f_1 .
2. Find the best-scoring candidate location that is not simply a restatement of the same location (typically: does not overlap the first by more than some fraction), score $f_2 \leq f_1$.
3. If f_1 and f_2 are close, the read could plausibly have come from either place — report low confidence. If f_2 is much worse than f_1 (or no second candidate exists at all), report high confidence.

shmap’s concrete formula, and the specific (somewhat unusual, all-or-nothing rather than smoothly graduated) way it implements step 3, is given in full in Chapter 12, including a discussion of two alternative formulas the original source retains, commented out, as evidence of in-progress tuning.

2.8 Summary of the theoretical pipeline

Putting the pieces together, shmap’s high-level strategy is: *FracMinHash sketch both sides for length-proportional, directly-comparable k -mer subsets; index the reference sketch once; for each read, seed rarest- k -mer-first against the index; accumulate hits into cheap, overlapping positional buckets; use the seed-heuristic upper bound to discard almost all buckets for a few arithmetic operations each; refine survivors with an exact sliding-window Jaccard/ containment sweep (or an LCS-based chain score); and report the best two candidates’ scores as a MAPQ estimate.* Chapter 3 now turns this into the concrete system architecture, and the chapters that follow specify every step precisely enough to implement.

Chapter 3

General Architecture

3.1 Two phases: indexing and mapping

shmap’s execution splits into two clearly separated phases, run once per invocation:

1. **Indexing phase** (Chapter 6): read the reference FASTA once, FracMinHash-sketch every segment, and build a hash map from k -mer hash to the list of reference positions (*hits*) sharing that hash. This is done once regardless of how many reads are mapped afterward.
2. **Mapping phase** (Chapters 7–10, reassembled end-to-end in Chapter 11): stream the read FASTA/FASTQ one record at a time; for each read, sketch it, seed it against the index, bucket the hits, prune buckets with the seed heuristic, refine survivors, compute MAPQ from the best two survivors, and emit one PAF line.

3.2 Configuration axes: the three compile-time flags

The reference C++ implementation parameterizes its core mapper class as a template, `SHMapper<no_bucket_pruning, one_sweep, abs_pos>`, over three independent booleans. In the original source these are *compile-time* template parameters (so that the compiler can specialize/inline branch-free code for whichever combination is selected at build time), but they are semantically just three independent feature toggles, exposed at the CLI as `-P`, `-B`, `-F` respectively:

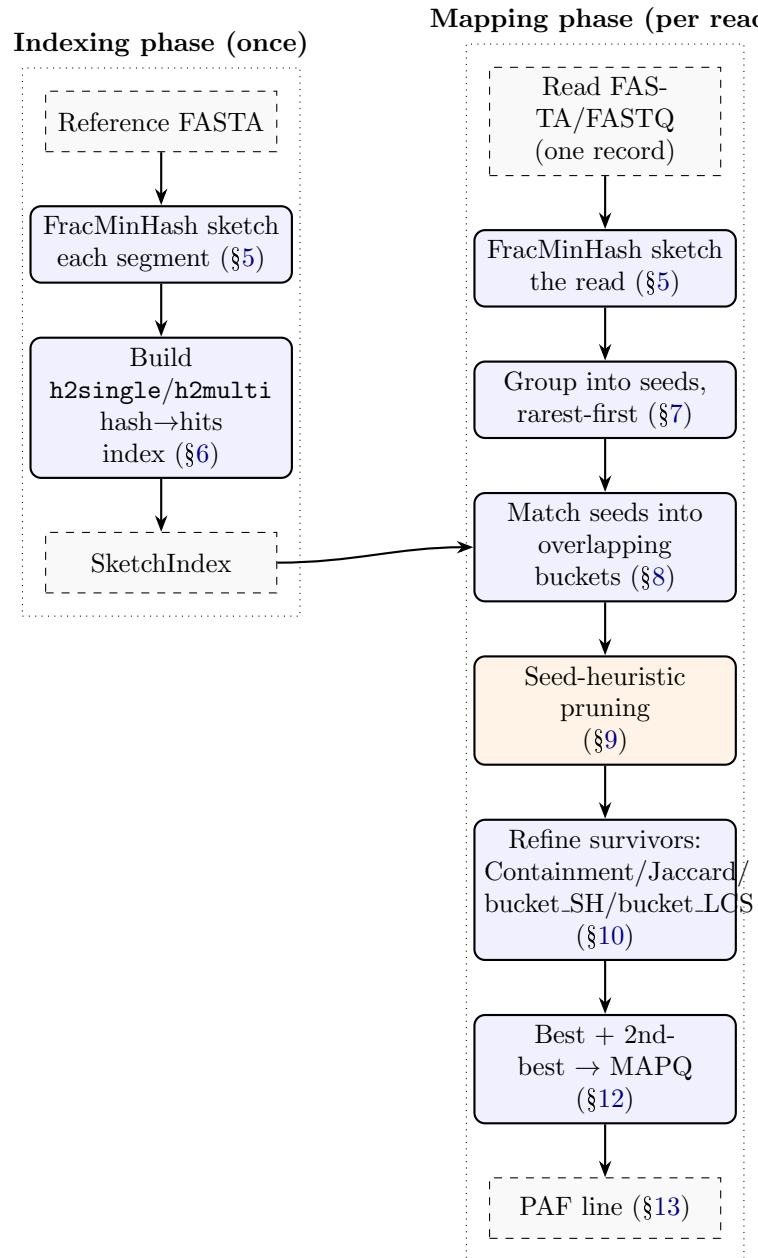


Figure 3.1: shmap’s two-phase pipeline. The indexing phase runs once; the mapping phase repeats for every read in the query file, consulting the already-built index.

Flag / parameter	Effect
<code>no_bucket_pruning</code> (-P)	When true, <i>disables</i> the seed-heuristic pruning of §2.6 entirely: every candidate bucket is fully scored. Correct but much slower; useful as a ground-truth oracle to verify pruning never discards a bucket it shouldn't.
<code>one_sweep</code> (-B)	Intended (per its own -B help text, “disregards seed heuristic and runs one sweep on all matches”) to skip the incremental per-bucket seed budget entirely and consider every seed at once. <i>Chapter 16 documents that in the reference source this flag is fully dead: parsed, templated, and printed, but never once branched on inside the mapper.</i>
<code>abs_pos</code> (-F)	Selects the coordinate space used for bucket indexing and window-size comparisons: <i>sketch index</i> (position within the array of sketch <i>k</i> -mers, i.e. “the <i>i</i> -th kept <i>k</i> -mer”) when false (default), or <i>absolute nucleotide position</i> when true. This changes bucket boundary arithmetic throughout Chapters 8–10 (every place that reads “ <i>in sketch-index units, or nucleotide units if abs_pos</i> ”).

Implementation note

A from-scratch implementation does not need compile-time specialization to get shmap's performance characteristics: these three booleans can equally well be ordinary runtime parameters threaded through the mapping functions (this is exactly what the accompanying Rust re-implementation does, using `const` generics only to keep the option of zero-cost specialization available, not because runtime dispatch is fundamentally required). What *is* essential to preserve is that each one is read consistently by *every* function that needs it — the original source's own bug catalogue (Chapter 16) includes at least one flag that looks load-bearing but silently isn't.

3.3 The metric axis

Orthogonal to the three structural flags above, a fourth, purely-scoring-level axis selects *how* a surviving bucket's similarity score is computed (CLI flag `-m`, four values): `Containment`, `Jaccard`, `bucket_SH`, `bucket_LCS`. These map directly onto the theory of §2.3.3 (containment/Jaccard) and §2.5 (LCS); `bucket_SH` is a degenerate/cheap option that reuses the seed-heuristic bound itself as the final score (no refinement sweep at all). Full formulas are in §??.

3.4 Data-flow summary: what crosses each module boundary

Stage	Input	Output
Sketch reference	raw nucleotide bytes per segment	<code>Vec<Kmer></code> per segment (position, hash, strand)
Build index	all segments' <code>Vec<Kmer></code>	<code>hash → Hit</code> (unique) or <code>hash → Vec<Hit></code> (repeated), each <code>Hit</code> carrying segment id + position
Sketch read	raw nucleotide bytes of the read	<code>Vec<Kmer></code>
Seed	<code>Vec<Kmer></code> of the read	<code>Vec<Seed></code> : one entry per distinct hash, with occurrence count in the read and reference hit count, sorted rarest-reference-hit-first
Bucket / match seeds	<code>Vec<Seed></code> (budget-limited prefix) + index	per-(segment, bucket) running <code>BucketContent</code> (seed count, match count, codirection, position range)
Prune	sorted buckets (most-matches-first) + remaining unconsumed seeds	surviving buckets only (most are discarded using $O(1)$ arithmetic per extension step)
Refine	one surviving bucket + the read's seed table	one <code>Mapping</code> (score, matched interval, strand, intersection count)
Best/second-best	every refined <code>Mapping</code>	the two highest, non-overlapping, <code>Mappings</code>
MAPQ	best two scores + same-strand-seed counts	integer MAPQ in $\{0, 5, 60\}$ (shmap's concrete range; see Chapter 12)
Output	final <code>Mapping</code> (+ globally accumulated counters)	one PAF text line with custom tags

3.5 Complexity at a glance

Let $n = |T|$ (total reference length), $m =$ a read's sketch size ($\approx r \cdot |P|$), and $S \leq m$ the seed budget actually consumed for that read (§7.2). Indexing is $O(n)$ time (one rolling-hash pass) and $O(rn)$ space (the kept fraction of k -mers). Per read: seeding/sorting is $O(m \log m)$; seed matching against the index is $O(S \cdot \bar{d})$ where $\bar{d}$ is the average number of reference hits per consumed seed (bounded above by $-M$, the max-matches-per-kmer cap, §6.2.3); bucket pruning is, in the typical case where most buckets are eliminated in $O(1)$ extensions, sublinear in the number of buckets touched; and refinement of each surviving bucket costs $O(|\text{bucket}|)$ (a single linear sweep). Full derivations are in Chapter 15.

Chapter 4

Core Data Types

This chapter specifies every value type used throughout the pipeline: its fields, the exact unit and range of each field, and the invariants callers may assume. All positions use **signed** 32-bit integers throughout the reference implementation (a legacy of an earlier dependency on `klib`, which does not support 64-bit positions) — a from-scratch implementation targeting references larger than 2^{31} bases should widen these, but should otherwise preserve signedness, since several computations (bucket predecessor indices, sentinel -1 values) rely on it.

4.1 Scalar type aliases

Alias	Underlying type	Meaning
<code>hash_t</code>	<code>uint64_t</code>	A k -mer’s (canonical, XOR-combined) hash value.
<code>rpos_t</code>	<code>int32_t</code>	A position in the <i>reference</i> : either a nucleotide offset or a sketch-array index, depending on context (see <code>abs_pos</code> , §3.2).
<code>qpos_t</code>	<code>int32_t</code>	A position/count in the <i>query</i> (read): nucleotide offset, sketch-array index, or a plain count (occurrence counts, seed budgets).
<code>segm_t</code>	<code>int32_t</code>	Index of a reference segment (0-based, in FASTA record order).

Implementation note

`rpos_t` and `qpos_t` are both plain 32-bit signed integers with no type-level distinction in the original C++ (they are both `typedefs` for the same underlying type) — the split is purely for reader clarity about which “space” a value lives in. A from-scratch implementation may use one integer type for both, but keeping the naming distinction in comments is strongly recommended: several bugs found in the reference source (Chapter 16) stem exactly from a value computed as a query-space quantity being fed into a reference-space-typed slot, or vice versa.

4.2 `Kmer`

A single sketched k -mer: its position, hash, and strand.

Field	Type	Meaning
<code>r</code>	<code>rpos_t</code>	Zero-based index of the k-mer’s last (rightmost) nucleotide in its source sequence. <i>Not</i> the exclusive right boundary of the half-open window, despite what an earlier version of the source’s own doc-comment claims — verified precisely in §5.3: for the i -th (0-based) k -mer of a sequence, $r = i + k - 1$.
<code>h</code>	<code>hash_t</code>	Canonical hash (XOR of forward and reverse-complement rolling hashes; §5.4).
<code>strand</code>	<code>bool</code>	<code>false</code> = forward strand “won” the hash comparison, <code>true</code> = reverse-complement did. See §5.4 for exactly how this is decided.

Equality on `Kmer` (where defined at all) compares *only* `h`; no code path in the reference implementation actually calls it, however (confirmed by exhaustive cross-reference during the Rust port) — every equality check in the real algorithm compares `.h` fields directly.

4.3 Hit

A `Kmer`’s occurrence *in the reference*, with its reference-relative metadata attached.

Field	Type	Meaning
<code>r</code>	<code>rpos_t</code>	Copied from the source <code>Kmer.r</code> (absolute nucleotide position of the hit’s last base within its segment).
<code>tpos</code>	<code>rpos_t</code>	Index of this hit <i>within the segment’s own sketch array</i> (i.e., “this is the <code>tpos</code> -th kept k -mer of this reference segment”).
<code>strand</code>	<code>bool</code>	Copied from the source <code>Kmer.strand</code> .
<code>segm_id</code>	<code>segm_t</code>	Which reference segment this hit belongs to.

Constructed as `Hit(kmer, tpos, segm_id)` — `r` and `strand` are always copied straight from the source `Kmer`; only `tpos` and `segm_id` are supplied independently.

4.4 Seed

One *distinct* k -mer hash from a read’s sketch, with everything the mapper needs to know about how it behaves both in the read and in the reference.

Field	Type	Meaning
<code>kmer</code>	<code>Kmer</code>	One representative occurrence of this hash in the read (position is that of the <i>first</i> , in sort order, occurrence — see §7.1).
<code>hits_in_t</code>	<code>rpos_t</code>	Total number of times this hash occurs <i>anywhere in the whole reference</i> (looked up via <code>SketchIndex::count</code> , §6.3). Zero if the hash never occurs in the reference at all.
<code>occs_in_p</code>	<code>qpos_t</code>	Number of times this hash occurs <i>in the read itself</i> (i.e., the multiplicity of a repeated k -mer within one read).
<code>seed_num</code>	<code>qpos_t</code>	0-based rank of this seed among all distinct seeds of the read, <i>after</i> sorting rarest-reference-hit-first (§??) — i.e., the order seeds are actually consumed in.
<code>pmatches</code>	<code>Vec<qpos_t></code>	<i>Every</i> position in the read where this hash occurs, sorted in decreasing order (a direct consequence of the grouping sort’s tie-break, §7.1). Used only by the <code>bucket_LCS</code> metric (§10.4).

4.5 Match

A pairing of one `Seed` (from the read) with one `Hit` (in the reference) that share a hash.

Field	Type	Meaning
<code>seed</code>	<code>Seed</code> (by value in C++; by reference in the Rust port, see below)	The matched read-side seed.
<code>hit</code>	<code>Hit</code>	The matched reference-side hit.

`Match` additionally exposes `codirection()`, returning `+1` if `seed.kmer.strand == hit.strand` (the read and reference agree on strand at this k -mer) and `-1` otherwise; summing this over a candidate’s matches is how strand (forward vs. reverse complement) is inferred for the final PAF strand column.

Implementation note

Ownership matters here. The C++ `Match` *copies* the entire matched `Seed` — including its heap-allocated `pmatches` vector — into itself, and `Match` objects are constructed in the thousands per read (once per (seed, hit) pair scanned during bucket collection and both refinement sweeps). This is a real, measurable cost in the reference binary. The Rust re-implementation instead gives `Match` a borrow, `seed: &'p Seed`, tied to the lifetime of the read-scoped seed table it is built from — every `Vec<Match>` in the algorithm is constructed, consumed, and dropped within the processing of a single read, so this lifetime is trivially satisfiable and eliminates the per-match deep copy entirely. A future from-scratch implementation in a garbage-collected language should simply share/reference the `Seed`, not clone it, for the same reason.

4.6 BucketLoc

The coordinate of one bucket: which segment, and which half-length slot.

Field	Type	Meaning
<code>segm_id</code>	<code>segm_t</code>	Reference segment index.
<code>b</code>	<code>rpos_t</code>	Bucket slot index; slot b spans reference positions $[b \cdot \ell, (b+2) \cdot \ell)$ where ℓ is the current read’s bucket half-length (§8.1).

The default/sentinel value is `(-1, -1)`, used to mean “no second-best bucket found yet” in output. This sentinel is part of the *observable output format* (it is printed literally as `(-1,-1)` in the `b2:s:` PAF tag when a read has no second-best mapping), so a from-scratch implementation that wants byte-compatible output should preserve it as a real sentinel value rather than, say, an optional/nullable type whose absence is rendered differently.

4.7 BucketContent

The running accumulator attached to each bucket as seeds are matched into it.

Field	Type	Default	Meaning
<code>i</code>	<code>int</code>	<code>-1</code>	Index of the next <i>seed</i> (into the read’s sorted seed list) to extend this bucket with, during incremental pruning (§9.3).
<code>seeds</code>	<code>qpos_t</code>	<code>0</code>	Seed <i>occurrences</i> attributed to this bucket so far (§2.6).
<code>matches</code>	<code>qpos_t</code>	<code>0</code>	Of those, how many actually landed a hit in this bucket.
<code>codirection</code>	<code>int</code>	<code>0</code>	Running sum of ± 1 per matched hit (§4.5); sign indicates the dominant strand.
<code>r_min, r_max</code>	<code>rpos_t</code>	<code>$+\infty, -1$</code>	Smallest/largest nucleotide position (<code>Hit.r</code>) seen among this bucket’s matches so far — used directly as the candidate’s reported boundary by the <code>bucket.SH</code> metric.

`BucketContent` supports an additive merge (`operator+=` in C++, an `AddAssign` impl in Rust) that sums `matches/seeds/codirection` and takes the componentwise min/max of the position range — used whenever two partial accumulations of the same bucket need combining (e.g. a hit contributing to both bucket *b* and its overlapping neighbor *b*−1, §8.1).

4.8 RefSegment

One indexed reference sequence (e.g. one chromosome/contig).

Field	Type	Meaning
<code>kmers</code>	<code>Vec<Kmer></code>	This segment’s full <code>FracMinHash</code> sketch, in increasing position order.
<code>name</code>	<code>String</code>	FASTA record id (used as the PAF target-name column and for ground-truth segment lookup, §13.3).
<code>sz</code>	<code>rpos_t</code>	Segment length in nucleotides (PAF target-length column).
<code>id</code>	<code>segm_t</code>	This segment’s own index (redundant with its position in the segment array, kept for convenience).

Implementation note

The C++ `RefSegment` additionally stores the segment’s full raw nucleotide sequence (`seq`), intended for a SAM/CIGAR alignment feature that is entirely commented out and unreachable in the shipped tool. Retaining it roughly doubles index memory for zero functional benefit; a from-scratch implementation targeting only the mapping (not alignment) functionality described in this document should omit it.

4.9 Hash-map type aliases

Four hash-map shapes recur throughout the algorithm; a from-scratch implementation may use any hash map with average $O(1)$ get/insert (the reference C++ uses two different fast open-addressing hash map libraries; the choice is a pure performance detail with no semantic effect):

Alias	Shape	Populated in / used by
<code>h2single</code>	<code>hash_t</code> $\rightarrow$ <code>Hit</code>	Reference index, k -mers with exactly one occurrence (§6).
<code>h2multi</code>	<code>hash_t</code> $\rightarrow$ <code>Vec<Hit></code>	Reference index, k -mers with ≥ 2 occurrences, each list sorted by (<code>segm_id</code> , <code>r</code>) for binary search (§6.2.2).
<code>h2seed_t</code>	<code>hash_t</code> $\rightarrow$ <code>Seed</code>	Per-read seed lookup table, built once per read from its unique seeds (§7.1).
<code>h2cnt</code>	<code>hash_t</code> $\rightarrow$ <code>qpos_t</code>	Per-read “remaining occurrence budget” table (<code>diff_hist</code>), decremented/incremented as a sliding window consumes/releases matches of a given hash during refinement (§10.1); values may legitimately go negative transiently (see that section for why).

4.10 Mapping and its stats sub-structures

A `Mapping` is the algorithm’s single “current best candidate” record: it behaves like a running maximum, only ever overwritten by `update(...)` when a strictly higher score is found (initial score is the sentinel -1.0 , guaranteed to lose to any real, non-negative similarity score). It bundles three logical groups of fields:

Group	Represents	Key fields
<code>MappingPaf</code>	The 12 mandatory PAF columns (§13)	<code>p_start</code> , <code>p_end</code> , <code>strand</code> , <code>s</code> , <code>s_sz</code> (matched reference span), <code>intersection</code> , <code>j/j2</code> (best/2nd best score), <code>sh</code> (seed-heuristic value at acceptance), <code>bucket/b</code>
<code>LocalMappingStats</code>	Per-read/per-candidate diagnostics, always populated	<code>seeds</code> , <code>total_matches</code> , <code>seeded_buckets</code> , <code>final_bucket</code> , <code>match_index</code>
<code>GlobalMappingStats</code>	Run-wide diagnostics, populated once per read only if a mapping was found	

Full field-by-field definitions, exact default/sentinel values, and the precise PAF/tag text each one is rendered as are given together with the output format in Chapter 13, since their only

purpose is serialization; the *algorithmic* methods that live on `Mapping` (`update`, `set_second_best`, `calc_mapq`, `overlap`) are specified where they are actually invoked: `update` in Chapter 10, `set_second_best/overlap` in Chapter 11, and `calc_mapq` in Chapter 12.

Implementation note

Two of `Mapping`'s stats fields deserve an early flag because they affect how a from-scratch implementer should read every later chapter's arithmetic: `intersection2` (second-best match count) and `bucket2` default to the sentinel values `-1` and `(-1, -1)` respectively, and *stay there* for any read with no viable second-best candidate — this is a normal, expected, frequently-occurring output state (a uniquely-mapping read), not an error state, and downstream formulas (notably `sigmas_diff` and `calc_mapq` in Chapter 12) are specifically written to treat the sentinel `-1` as “0” rather than its literal value.

Chapter 5

FracMinHash Sketching

This chapter specifies `FracMinHash::sketch`, the single most bit-sensitive routine in the whole system: every downstream index, seed, and score depends on hashes being computed exactly this way, since reference and read sketches must use an *identical* hash function to be comparable at all.

5.1 Inputs and outputs

	Type	Meaning
<i>Input</i> s	raw ASCII nucleotide bytes (upper- or lower-case A/C/G/T; anything else, see §5.7)	The sequence to sketch: one reference segment, or one read.
<i>Config</i> k	positive integer	k -mer length (CLI <code>-k</code> , default 15).
<i>Config</i> h_frac	$r \in (0, 1]$	Keep-fraction of the hash space (CLI <code>-r</code> , default 0.05).
<i>Output</i>	<code>Vec<Kmer></code>	Every k -mer of s whose canonical hash clears the threshold, in increasing position order, each tagged with its hash and winning strand (§4.2). Empty if $ s < k$.

5.2 Lookup tables: per-nucleotide hash contributions

The rolling hash is built from two 256-entry lookup tables, `LUT_fw` and `LUT_rc`, indexed by raw ASCII byte value, giving each of the four nucleotides (in both upper and lower case) a fixed 64-bit contribution:

Base	LUT_fw contribution (hex)	LUT_rc contribution
A/a	0x3c8bfbb395c60474	= <code>LUT_fw['T']</code>
C/c	0x3193c18562a02b4c	= <code>LUT_fw['G']</code>
G/g	0x20323ed082572324	= <code>LUT_fw['C']</code>
T/t	0x295549f54be24456	= <code>LUT_fw['A']</code>

That is, a base’s reverse-complement contribution is defined as its *complementary* base’s forward contribution — exactly encoding the biological complement rule ($A \leftrightarrow T$, $C \leftrightarrow G$) at the hash-contribution level, so that no explicit “complement a character” step is ever needed; complementation falls out of table lookup alone. All other byte values (N, ambiguity codes, whitespace, etc.) have *no defined contribution* in the reference source.

Known defect in the reference implementation

The reference C++ declares `LUT_fw/LUT_rc` as plain fixed-size arrays and initializes only the 8 entries above; the remaining 248 entries are **uninitialized stack memory** at the time `sketch()` first reads them. For any input byte other than `ACGTacgt` (most commonly `N`), the hash computation silently incorporates garbage. This is undefined behavior, not merely “an unspecified but fixed value” — the actual bits read depend on whatever happened to be on the stack. **Recommended fix for a from-scratch implementation:** zero-initialize both tables before filling in the 8 real entries, so that any non-ACGT byte deterministically contributes a hash of 0. This changes no behavior for any ACGT-only input (which is the overwhelming majority of real sequence data) and removes the UB.

5.3 The position semantics of `Kmer.r`

Before giving the recurrence, it is worth nailing down precisely what value ends up in a produced `Kmer`’s `r` field, since it is easy to get this off by one and it is exercised by the reference test suite. For the (0-based) i -th k -mer of a sequence (spanning nucleotide positions $[i, i+k)$):

$$\text{Kmer.r} = i + k - 1$$

i.e., **the index of the k -mer’s last nucleotide**, not the exclusive upper bound of its span (despite a doc-comment in the reference source claiming the latter). This is verified by the reference test suite itself: for $k = 4$ and a 5-character sequence, the two produced k -mers (at $i = 0, 1$) have `r = 3, 4` respectively, matching $i + k - 1$.

5.4 The rolling-hash recurrence

Let s be the input, k the k -mer length, and define, for a byte c , $\text{fw}(c) = \text{LUT_fw}[c]$ and $\text{rc}(c) = \text{LUT_rc}[c]$. The threshold is

$$H_{\text{thr}} = \begin{cases} \lfloor r \cdot (2^{64} - 1) \rfloor & r < 1 \\ 2^{64} - 1 & r = 1 \end{cases}$$

computed once via a floating-point multiply-then-truncate (this is the one place floating-point rounding can matter at all — for $r = 1$ it is special-cased to the exact maximum, avoiding any rounding).

Initialization (first window, $i = 0$). Seed $h_{fw} = h_{rc} = 0$, then fold in each of the first k bases with a position-dependent rotation:

$$h_{fw} \oplus= \text{rotr}(\text{fw}(s[j]), k - j - 1), \quad h_{rc} \oplus= \text{rotr}(\text{rc}(s[j]), j) \quad \text{for } j = 0, \dots, k - 1.$$

($\text{rotr}(x, n)$ / $\text{rotr}(x, n)$ denote 64-bit bitwise left/right rotation.) Note the two hashes rotate in *opposite* directions and by *mirrored* amounts ($k-j-1$ vs. j) — this is precisely what makes h_{rc} equal to the forward hash of the reverse-complement string: reversing the string reverses which position gets which rotation amount, and complementing each base is handled already by using `LUT_rc` instead of `LUT_fw`.

Emitting a k -mer. At every window position (including the very first, before any roll), combine the two sides and test the threshold:

$$h = h_{fw} \oplus h_{rc}, \quad \text{keep iff } h \leq H_{\text{thr}}.$$

If kept, its strand bit is $\mathbf{strand} = (h_{fw} > h_{rc})$ — i.e., whichever side is numerically larger is deemed to have “won” and determines orientation. (This is an arbitrary but self-consistent convention: swap the comparison and every strand bit flips consistently, with no effect on any similarity computation, since those only ever compare two *Kmers*’/ *Hits*’ strand bits to each other, never to an absolute notion of “forward”.)

Rolling to the next window ($i \rightarrow i + 1$, incoming base $s[i+k]$, outgoing base $s[i]$).

$$h_{fw} \leftarrow \text{rotl}(h_{fw}, 1) \oplus \text{rotl}(\text{fw}(s[i]), k) \oplus \text{fw}(s[i+k])$$

$$h_{rc} \leftarrow \text{rotr}(h_{rc}, 1) \oplus \text{rotr}(\text{rc}(s[i]), 1) \oplus \text{rotl}(\text{rc}(s[i+k]), k - 1)$$

Each update: (1) rotates the whole accumulated hash by one position in the direction that side rolls, (2) removes the outgoing base’s contribution (itself rotated to the position it now needs to cancel from), (3) adds the incoming base’s contribution at its new position. This is the standard structure of an XOR/rotate rolling hash: rotation distributes each base’s contribution across all 64 bits over the k -window it participates in, and since rotl/rotr are self-inverse under repeated application by the same total, the outgoing base’s contribution cancels *exactly* (XOR with itself) once it has rolled all the way through.

Algorithm: title=sketch(s, k, h_frac)

```

if len(s) < k: return []
H_thr := (h_frac < 1) ? floor(h_frac * (2^64 - 1)) : 2^64 - 1
h_fw, h_rc := 0, 0
for j in 0..k-1:
    h_fw ^= rotl(LUT_fw[s[j]], k-j-1)
    h_rc ^= rotl(LUT_rc[s[j]], j)
r := k - 1                # position of the k-mer's last base, 0-based
result := []
loop:
    h := h_rc ^ h_fw
    if h <= H_thr:
        strand := (h_fw > h_rc)
        result.append(Kmer{ r: r, h: h, strand: strand })
    if r + 1 >= len(s): break
    out_c, in_c := s[r + 1 - k], s[r + 1]
    h_fw := rotl(h_fw, 1) ^ rotl(LUT_fw[out_c], k) ^ LUT_fw[in_c]
    h_rc := rotr(h_rc, 1) ^ rotr(LUT_rc[out_c], 1) ^ rotl(LUT_rc[in_c], k-1)
    r := r + 1
return result

```

Implementation note

The loop above is written with the loop variable r directly holding the emitted *Kmer*. r value at every iteration (equivalent to, but clearer than, the reference source’s own `for(r=0;...)` ... `push(Kmer(r-1,...))` formulation, which tracks “one past” internally and subtracts 1 only at emission time). Both are exactly equivalent; verify against §5.3 if re-deriving independently.

5.5 Reference C++ implementation

```

1  const sketch_t sketch(const std::string& s) const {
2      sketch_t kmers;
3      kmers.reserve((rpos_t)(1.1 * (double)s.size() * hFrac));
4      if (rpos_t(s.size()) < k) return kmers;
5
6      hash_t h_fw = 0, h_rc = 0;
7      hash_t hThres = hFrac < 1.0
8          ? hash_t(hFrac * double(std::numeric_limits<hash_t>::max()))
9          : std::numeric_limits<hash_t>::max();
10     rpos_t r;
11
12     for (r=0; r<k; r++) {
13         h_fw ^= std::rotl(LUT_fw[ size_t(s[r]) ], k-r-1);
14         h_rc ^= std::rotl(LUT_rc[ size_t(s[r]) ], r);
15     }
16
17     while(true) {
18         auto h = h_rc ^ h_fw;
19         if (h <= hThres) {
20             const bool strand = h_fw > h_rc;
21             kmers.push_back(Kmer(r-1, h, strand));
22         }
23         if (r >= rpos_t(s.size())) break;
24         h_fw = std::rotl(h_fw, 1) ^ std::rotl(LUT_fw[ size_t(s[r-k]) ], k) ^ LUT_fw[ size_t(s[r]) ];
25         h_rc = std::rotr(h_rc, 1) ^ std::rotr(LUT_rc[ size_t(s[r-k]) ], 1)
26             ^ std::rotl(LUT_rc[ size_t(s[r]) ], k-1);
27         ++r;
28     }
29     return kmers;
30 }

```

Listing 5.1: FracMinHash::sketch, src/sketch.h (verbatim)

5.6 Rust re-implementation

```

1  pub fn sketch(&self, s: &[u8], counters: &mut Counters) -> SketchT {
2      let k = self.k;
3      let mut kmers: SketchT =
4          Vec::with_capacity((1.1 * s.len() as f64 * self.h_frac).max(0.0) as usize);
5      if (s.len() as RPos) < k { return kmers; }
6
7      let h_thres: Hash = if self.h_frac < 1.0 {
8          (self.h_frac * u64::MAX as f64) as u64
9      } else { u64::MAX };
10
11     let mut h_fw: Hash = 0;
12     let mut h_rc: Hash = 0;
13     let mut r: RPos = 0;
14     while r < k {
15         let c = s[r as usize] as usize;
16         h_fw ^= self.lut_fw[c].rotate_left((k - r - 1) as u32);
17         h_rc ^= self.lut_rc[c].rotate_left(r as u32);
18         r += 1;
19     }
20
21     loop {
22         let h = h_rc ^ h_fw;
23         if h <= h_thres {
24             let strand = h_fw > h_rc;
25             kmers.push(Kmer::new(r - 1, h, strand));
26         }
27         if r >= s.len() as RPos { break; }
28         let out_c = s[(r - k) as usize] as usize;
29         let in_c = s[r as usize] as usize;
30         h_fw = h_fw.rotate_left(1) ^ self.lut_fw[out_c].rotate_left(k as u32) ^ self.lut_fw[in_c];
31         h_rc = h_rc.rotate_right(1)
32             ^ self.lut_rc[out_c].rotate_right(1)
33             ^ self.lut_rc[in_c].rotate_left((k - 1) as u32);
34         r += 1;

```

```

35     }
36     kmers
37 }

```

Listing 5.2: `FracMinHash::sketch`, Rust port (`src/sketch.rs`)

Note the Rust version zero-initializes both LUTs before filling the 8 real entries (the fix from the bug box above); everything else is a direct, bit-for-bit translation — `rotate_left/rotate_right` on a `u64` are exactly `std::rotl/std::rotr` on `uint64_t`, including the fact that both are defined for *any* shift amount (taken mod 64), so no extra bounds-masking is needed even though k can exceed typical small shift-amount assumptions.

5.7 Non-ACGT input

Real sequencing data occasionally contains `N` (unknown base) or, rarely, IUPAC ambiguity codes. As established above, the reference implementation’s behavior on these is undefined; the recommended, safe, deterministic choice — adopted by the Rust port — is that such a byte contributes a hash of exactly 0 on both the forward and reverse-complement side. This is a reasonable default because it is simple, deterministic, and does not privilege any particular real base; a from-scratch implementation is free to choose a different deterministic convention (e.g. skip k -mers containing any non-ACGT byte entirely, which several other tools do) as long as it is applied *identically* when sketching the reference and the reads, since any asymmetry there would silently break matching.

5.8 Complexity

Sketching a sequence of length n takes $O(n)$ time (one $O(1)$ -amortized rolling-hash step per position, after an $O(k)$ initialization) and produces, in expectation, $O(rn)$ output k -mers, using $O(rn)$ space for the output (input is streamed, not buffered beyond what the caller already holds).

5.9 Worked example

For $k = 4$, $r = 1.0$ (keep everything) and input "ACGGT" (length 5, so 2 k -mers), the two produced `Kmers` have `r = 3, 4` (matching $i + k - 1$ for $i = 0, 1$). Feeding the reverse complement of the same underlying sequence, "ACCGT", produces the *same multiset of hashes* in reversed position order and with each `strand` bit flipped relative to the original — this symmetry (verified as an automated test in both the C++ and Rust implementations) is the single most important sanity check to run against any from-scratch reimplementations before trusting it: sketch a string and its reverse complement, reverse one result, and confirm the hash sequences match exactly.

Chapter 6

Reference Indexing

The index (`SketchIndex`) is built exactly once per run, from the reference FASTA, before any read is processed. Its job is to answer, in $O(1)$ average time, two questions for any k -mer hash h : *how many times does h occur in the reference* (§6.3), and *at which (segment, position) pairs* (used throughout bucketing/scoring).

6.1 Data owned by the index

Field	Type	Meaning
<code>segments</code>	<code>Vec<RefSegment></code>	One entry per FASTA record, in file order (§4.8).
<code>h2single</code>	<code>hash_t → Hit</code>	k -mers occurring <i>exactly once</i> across the whole reference.
<code>h2multi</code>	<code>hash_t → Vec<Hit></code>	k -mers occurring <i>two or more</i> times, each list sorted (§6.2.2).

Splitting single- vs. multi-occurrence k -mers into two separate maps (rather than one map to always-a-list) is a memory/speed optimization: the overwhelming majority of k -mers in a non-repetitive genome occur exactly once, so storing those as a bare `Hit` (no vector indirection/allocation) rather than a one-element `Vec<Hit>` meaningfully reduces both memory and pointer-chasing.

6.2 Building the index: `build_index`

Algorithm: `title=build_index(t_file, k, h_frac, max_matches)`

```
for (segm_name, seq) in read_fasta(t_file):
    sketch := FracMinHash.sketch(seq, k, h_frac)           # Chapter 5
    segm_id := len(segments)
    segments.append(RefSegment{ kmers: sketch, name: segm_name,
                                sz: len(seq), id: segm_id })
    populate_h2pos(sketch, segm_id)                         # below

# migrate any hash that ended up in BOTH maps fully into h2multi
for (h, hits) in h2multi:
    if h in h2single:
        hits.append(h2single.remove(h))
    sort hits by (segm_id, r)                               # enables binary search later

if max_matches is set:
    erase_frequent_kmers(max_matches)                       # below
```

6.2.1 populate_h2pos: routing each hit to the right map

Algorithm: title=populate_h2pos(sketch, segm_id)

```
for tpos, kmer in enumerate(sketch):
    hit := Hit(kmer, tpos, segm_id)
    if kmer.h not in h2single:
        h2single[kmer.h] := hit                # first-ever occurrence: goes to h2single
    else:
        # second-or-later occurrence: goes to h2multi (h2single's own
        # earlier entry is migrated into h2multi at the *end* of indexing,
        # not immediately -- see build_index above)
        if max_matches is unset or len(h2multi[kmer.h]) < max_matches + 1:
            h2multi[kmer.h].append(hit)
```

Implementation note

Note the asymmetry this creates *during* indexing (before the end-of-build_index migration step runs): a hash with exactly 2 occurrences temporarily has its *first* occurrence sitting in h2single and only its *second* occurrence in h2multi — h2multi[h] would report length 1, not 2, if queried mid-build. This is why the migration step (moving h2single's entry into h2multi and *erasing* it from h2single) must run once, after all segments have been sketched, before the index is considered complete and used for querying.

The max_matches + 1 cap during population (rather than exactly max_matches) is deliberate slack: it lets erase_frequent_kmers (below) later distinguish “exactly at the cap” from “over the cap” after the single-entry migration adds one more; in practice, since erase_frequent_kmers deletes any hash whose *final* count exceeds max_matches outright, the effect is simply an upper bound on wasted work spent appending hits for a hash that is going to be discarded anyway.

6.2.2 Sorting multi-hit lists

Each h2multi list is sorted by (segm_id, r) lexicographically. Because *r* (a hit's nucleotide position) and *tpos* (its index into the segment's own sketch array) increase together within one segment (the sketch is built and stored in increasing position order, §4.8), sorting by *r* *also* sorts by *tpos* consistently — so binary search by either coordinate space works correctly against the same sorted list, which matters because bucket/window boundary checks are expressed in *either* space depending on the abs_pos flag (§3.2).

6.2.3 erase_frequent_kmers: capping pathologically repetitive *k*-mers

CLI flag -M sets max_matches, an optional hard cap on how many reference occurrences a single *k*-mer hash is allowed to have before it is discarded from the index entirely (treated thereafter as if it never occurred in the reference at all, i.e. hits_in_t = 0 for it). This exists because extremely repetitive *k*-mers (satellite repeats, low-complexity runs) contribute enormous numbers of hits that are almost never informative for mapping but are expensive to iterate during seeding/bucketing — capping them is a standard technique shared with essentially every other *k*-mer-based mapper (minimap2's -f, Winnowmap's repetitive-*k*-mer blacklist, etc.).

Algorithm: title=erase_frequent_kmers(max_matches)

```
for (h, hits) in h2multi:
    if len(hits) > max_matches:
        delete h2multi[h]
```

Note this only prunes `h2multi` (by construction, `h2single` entries always have exactly one occurrence, so they can never exceed any cap ≥ 1).

6.3 `count(h)`: reference occurrence count for a hash

Algorithm: `title=count(h)`

```
if h in h2single: return 1
if h in h2multi:  return len(h2multi[h])
return 0
```

This is the function seeding (Chapter 7) calls once per distinct read k -mer to determine `hits_in_t` (§4.4) — it is the quantity seeds are sorted by (rarest first) and the input to the seed-heuristic bound’s implicit assumption that `hits_in_t` = 0 seeds contribute nothing.

6.4 Reference C++ implementation

```
1 inline rpos_t count(hash_t h) const {
2     if (auto it = h2single.find(h); it != h2single.end()) return 1;
3     if (auto it = h2multi.find(h); it != h2multi.end()) return it->second.size();
4     return 0;
5 }
6
7 void populate_h2pos(const sketch_t& sketch, segm_t segm_id) {
8     for (size_t tpos = 0; tpos < sketch.size(); ++tpos) {
9         const Kmer& kmer = sketch[tpos];
10        const auto hit = Hit(kmer, tpos, segm_id);
11        if (!h2single.contains(kmer.h))
12            h2single[kmer.h] = hit;
13        else if (H->params.max_matches == -1
14            || (rpos_t)h2multi[kmer.h].size() < H->params.max_matches + 1)
15            h2multi[kmer.h].push_back(hit);
16    }
17 }
18
19 void build_index(const std::string &tFile) {
20     read_fasta_klib(tFile, [this](const string &segm_name, const string &T, float) {
21         sketch_t t = H->sketcher.sketch(T);
22         add_segment(segm_name, T, t); // pushes RefSegment, calls populate_h2pos
23     });
24
25     for (auto &[h, hits] : h2multi) {
26         if (h2single.contains(h)) {
27             hits.push_back(h2single.at(h));
28             h2single.erase(h);
29         }
30         sort(hits.begin(), hits.end(), [](const Hit &a, const Hit &b) {
31             if (a.segm_id != b.segm_id) return a.segm_id < b.segm_id;
32             return a.r < b.r;
33         });
34     }
35     if (H->params.max_matches != -1) erase_frequent_kmers();
36 }
```

Listing 6.1: Core of SketchIndex, `src/index.h`

6.5 Complexity and memory

Indexing costs $O(n)$ time for sketching (Chapter 5) plus $O(rn \log d)$ for sorting, where rn is the total number of kept k -mers and d is the maximum multiplicity of any single hash (the sort is per-hash, so the $\log d$ factor applies independently to each list, not globally). Memory is

$O(rn)$: one map entry per distinct occurrence, plus $O(rn)$ for the stored per-segment sketches themselves (each `RefSegment.kmers`).

Chapter 7

Seeding

Seeding turns a read’s raw sketch (a flat list of k -mers, with repeats) into an ordered list of *seeds* — one entry per distinct hash — ready to be consumed budget-first against the index. This chapter covers `unique_elements_with_info` (grouping) and the derivation of the seed budget S used by the caller.

7.1 Grouping: `unique_elements_with_info`

Input. `p`: the read’s sketch, `Vec<Kmer>` (Chapter 5), mutable (it is sorted in place). *Implicit input*: the reference index, queried once per distinct hash.

Output. `Vec<Seed>` (§4.4), one per distinct hash value present in `p`, **sorted ascending by `hits_in_t`** (rarest-in-reference first). Also has the side effect of updating the `kmers_notmatched` diagnostic counter (§7.1.1 documents a bug in how the reference computes this specific counter — it does not affect any other output).

Algorithm: `title=unique_elements_with_info(p)`

```
# Step 1: group equal hashes together, and within a group, order by
# *decreasing* read position (this ordering becomes each seed’s pmatches
# list, which the bucket_LCS metric consumes directly, see 10.4).
sort p by (h ascending, r descending)

# Step 2: walk the sorted list, closing out a group whenever the hash changes.
p_unique := []
strike := 0                # occurrences seen in the current group so far
matches := []              # positions seen in the current group so far
for pos in 0 .. len(p)-1:
    strike += 1
    matches.append(p[pos].r)
    if pos == len(p)-1 or p[pos].h != p[pos+1].h:
        hits_in_t := index.count(p[pos].h)                # Section 6.4
        p_unique.append(Seed{
            kmer: p[pos], hits_in_t: hits_in_t,
            occs_in_p: strike, seed_num: len(p_unique),
            pmatches: copy(matches),
        })
        strike := 0
        matches := []
```

```
# Step 3: seeds are consumed rarest-reference-hit-first.
sort p_unique by hits_in_t ascending
return p_unique
```

Why sort by hits_in_t ascending. A k -mer that occurs rarely in the reference is far more discriminative than a common one: matching it narrows the candidate set of buckets dramatically, whereas matching a k -mer with thousands of reference occurrences adds thousands of (mostly irrelevant) candidate hits for very little narrowing benefit. Consuming seeds rarest-first under a fixed budget S (§7.2) maximizes the discriminative value extracted from that budget.

Why sort the group by decreasing position. `Seed.pmatches` ends up populated in the same order the group was scanned, i.e. decreasing read position — this specific order is required by `lcs_get_lis` (§10.4), which needs `pmatches` in a defined order to reconstruct multiplicities correctly across repeated seeds when computing a longest increasing subsequence.

7.1.1 Reference C++ implementation

```
1 Seeds unique_elements_with_info(sketch_t& p) {
2     sort(p.begin(), p.end(), [](const Kmer &a, const Kmer &b) {
3         if (a.h != b.h) return a.h < b.h;
4         return a.r > b.r; // reverse order of inclusion; needed for LCS
5     });
6
7     Seeds p_unique;
8     qpos_t strike = 0;
9     vector<qpos_t> matches;
10    for (size_t ppos = 0; ppos < p.size(); ++ppos) {
11        ++strike;
12        matches.push_back(p[ppos].r);
13        if (ppos == p.size()-1 || p[ppos].h != p[ppos+1].h) {
14            rpos_t hits_in_t = tidx.count(p[ppos].h);
15            p_unique.emplace_back(p[ppos], hits_in_t, strike, p_unique.size(), matches);
16            strike = 0;
17            matches.clear();
18        }
19    }
20
21    sort(p_unique.begin(), p_unique.end(), [](const Seed &a, const Seed &b) {
22        return a.hits_in_T < b.hits_in_T;
23    });
24    return p_unique;
25 }
```

Listing 7.1: `unique_elements_with_info`, `src/shmap.h`

Known defect in the reference implementation

The reference C++ additionally maintains a `nonzero` counter meant to track how many *read positions* (counting multiplicity) belong to seeds that do have at least one reference hit, so that the diagnostic `kmers_notmatched` stat can report “sketch size minus matched positions”. The increment is written as `if (hits_in_t > 0) nonzero += strike;` placed *after* `strike` has already been reset to 0 for the next group two lines above it — so it always adds 0, and `kmers_notmatched` always reports the entire sketch size regardless of how many k -mers actually matched. This affects *only* a cosmetic, end-of-run diagnostic percentage; it has no effect on any mapping decision, score, or PAF field. A from-scratch implementation should compute the increment *before* resetting `strike` (or reorder the two statements), which is the obviously-intended behavior.

7.2 The seed budget S

Not every seed is necessarily consumed — seeding stops once a budget S of seed *occurrences* (not distinct seeds; a repeated k -mer with `occs_in_p=3` counts as 3 against the budget when consumed, all at once, §8.4) has been spent. S is derived once per read from the acceptance threshold θ (CLI `-t`) and the minimum-score-gap parameter (CLI `-d`, used here as a safety margin, not in its other role computing the second-best threshold in Chapter 11):

$$\theta_2 = \theta - \text{min_diff}, \quad S = \lfloor (1 - \theta_2) \cdot m \rfloor + 1$$

where m is the read’s full sketch size. **Interpretation:** S is exactly the smallest seed-occurrence budget for which it is still *possible*, in principle, for a bucket to reach score θ_2 even if literally every consumed seed occurrence that has a reference hit at all ends up matching the same bucket: the seed-heuristic bound (§2.6) for a bucket that has consumed all S seed occurrences and matched every one of them is exactly $h_{\text{seed}} = 1 - 0/m$, and the bound only starts to allow room for *misses* once more than $(1 - \theta_2)m$ occurrences have been consumed — so S is the minimum budget that gives the pruning bound any chance of staying above θ_2 for a maximally-good bucket, with +1 guaranteeing at least one seed is always consumed even when $\theta_2 \rightarrow 1$.

Implementation note

This is a heuristic sizing choice, not a hard correctness requirement in either direction: seeds *beyond* S might still have found valuable matches (this is a real, accepted recall/speed trade-off, and is exactly the knob CLI flag `-t` tunes indirectly through θ_2); a from-scratch implementation experimenting with alternative sizing should keep in mind that S only bounds how many seed *occurrences* are matched against the index at all — it does not, by itself, bound the pruning behavior described in Chapter 9, which operates independently per bucket using whatever prefix of seeds *has* been consumed so far.

7.3 `more_seeds_if_cheap`: opportunistic budget extension (dead code)

Algorithm: `title=more_seeds_if_cheap(S, seeds)`

```
original_S := S
while S < len(seeds) and seeds[S].hits_in_t <= 1:
    S += 1
return S
```

This extends the budget past S for free whenever the next seed(s) are (almost) unique in the reference (`hits_in_t ≤ 1`): such seeds are essentially free to consume (at most one reference hit to check) and maximally discriminative, so there’s no real cost/benefit trade-off in including them.

Known defect in the reference implementation

This function is fully implemented and unit-testable, but its only call site in the reference C++ `map_read` is commented out (`//S = more_seeds_if_cheap(S, p.unique);`) — it is never actually invoked by the shipped tool. A from-scratch implementation may freely choose to wire it in (it is a strict improvement in recall at negligible extra cost) or leave it unused to match the reference binary’s actual observed behavior exactly.

Chapter 8

Bucketing

Bucketing is shmap’s substitute for full co-linear chaining (§2.4): instead of clustering individual hits by a general chaining algorithm, the reference is pre-partitioned into fixed-geometry, *overlapping* windows (*buckets*), and every hit is attributed to the bucket(s) whose window contains it. This section defines the bucket geometry precisely, then the two data structures that accumulate hits into buckets (`Buckets`, the primary store, and `BucketsHash`, an ephemeral scratch structure), then `match_seeds`, the procedure that actually populates them from a read’s seed list.

8.1 Bucket geometry

Fix a **half-length** ℓ (per read; §8.1.1). Bucket slot b (an integer, possibly negative only in the sense that $b \geq 0$ is enforced but not derived that way directly) spans the half-open interval

$$[b \cdot \ell, (b + 2) \cdot \ell)$$

in whichever coordinate space is active (§3.2: nucleotide position if `abs_pos`, else index into the segment’s sketch array). **Consecutive buckets overlap by exactly half their width:** bucket b and bucket $b+1$ share the sub-range $[(b+1)\ell, (b+2)\ell)$. A position p maps to its *primary* bucket via $b = \lfloor p/\ell \rfloor$; because of the 50% overlap, p is *also* inside bucket $b-1$ whenever $b > 0$ (since bucket $b-1$ ’s span $[(b-1)\ell, (b+1)\ell)$ still contains any p with $b\ell \leq p < (b+1)\ell$).

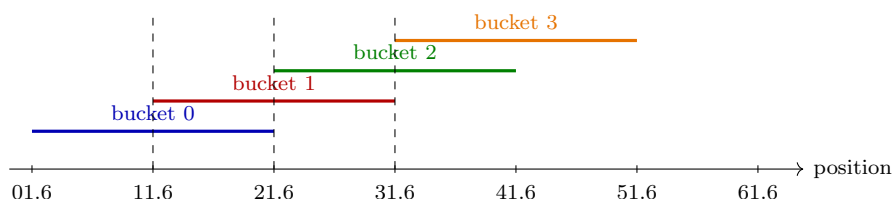


Figure 8.1: Bucket geometry: each bucket spans 2ℓ , consecutive buckets overlap by ℓ (50%). A hit at position $p \in [\ell, 2\ell)$ (e.g. the overlap region between buckets 0 and 1) is attributed to *both* bucket 0 and bucket 1.

Why 50% overlap. Without overlap, a true alignment straddling a bucket boundary would have its matches split roughly in half between two non-overlapping buckets, and neither alone would score well even though the true alignment is real and strong. With 50% overlap, *any* true alignment of length up to ℓ is guaranteed to be fully contained within at least one bucket (the one aligned with its start, or its neighbor), at the cost of roughly doubling the number of (hit, bucket) attributions that must be tracked. This is directly analogous to why minimizer schemes (§2.3.1) guarantee window coverage — it is the same “don’t let a real signal fall in a seam” concern, solved geometrically instead of via a window-based sampling guarantee.

8.1.1 Choosing ℓ : the bucket half-length

ℓ is set once per read, to the read's own sketch size m (or, under `abs_pos`, the read's raw nucleotide length) — i.e., **a bucket is sized to be exactly as wide as the read it's being used for**, so that a bucket can plausibly contain one full copy of the read's alignment. A minimum half-length $\ell_{\min} = 5$ is enforced as a sanity floor (extremely short reads/sketches are rejected as unmappable rather than creating degenerate buckets).

8.2 Buckets: the primary per-run accumulator

One `Buckets` instance is created once per mapping run (not per read) and `clear()`'d between reads (§8.2.1) — this reuses its backing storage rather than reallocating per read.

Field	Type	Meaning
<code>halflen</code>	<code>int</code>	Current read's ℓ (§8.1.1); reset every read.
<code>i</code>	<code>int</code>	Index of the next seed (into the current read's seed list) not yet propagated to every touched bucket's own <code>.i</code> counter (§8.2.3).
<code>seeds</code>	<code>int</code>	Seed-occurrence budget consumed so far, mirrored into every touched bucket.
<code>buckets</code>	one <code>Vec<BucketContent></code> per segment	Flat, directly-indexed storage: <code>buckets[segm_id][b]</code> is that bucket's running <code>BucketContent</code> (§4.7), pre-sized to $\lceil \text{segment_length} / \ell_{\min} \rceil + 2$ slots at construction time so that it is large enough for <i>any</i> valid $\ell \geq \ell_{\min}$ used later.
<code>non_empty_buckets_with_repeats</code>	<code>Vec<BucketLoc></code>	Every <code>BucketLoc</code> touched since the last <code>clear()</code> , with duplicates (a bucket touched by 5 different hits appears 5 times) — deduplicated only when <code>get_sorted_buckets</code> is called (§8.2.4). Doubling as the undo-log for <code>clear()</code> .

Implementation note

The reference C++ caps the number of reference segments at a hardcoded constant (`MAX_SEGMENTS = 100`), throwing at construction time for larger references. This reads as an arbitrary implementation limit, not an intentional one — draft/scaffolded genome assemblies routinely have thousands of small contigs. A from-scratch implementation should size `buckets` to the reference's *actual* segment count instead of imposing any fixed cap.

8.2.1 clear()

Algorithm: title=Buckets::clear()

```
i := 0
seeds := 0
for (segm_id, b) in non_empty_buckets_with_repeats:
    buckets[segm_id][b] := BucketContent::default() # reset just the touched slots
non_empty_buckets_with_repeats.clear()
```

Only slots that were actually touched are reset (tracked via the repeats log) — this is $O(\text{touched slots in the previous read})$, not $O(\text{reference size})$, which matters because the vast majority of a genome-scale reference’s buckets are never touched by any single read.

8.2.2 add_to_pos and add_to_bucket

Algorithm: title=add_to_pos(hit, content) / add_to_bucket(loc, content)

```
function add_to_pos(hit, content):
    b := (abs_pos ? hit.r : hit.tpos) / halfflen # integer division
    buckets[hit.segm_id][b] += content # BucketContent's additive merge
    log(hit.segm_id, b)
    if b > 0:
        buckets[hit.segm_id][b-1] += content # 50% overlap: also touches prede
        log(hit.segm_id, b-1)

function add_to_bucket(loc, content):
    buckets[loc.segm_id][loc.b] += content
    log(loc.segm_id, loc.b)
```

add_to_pos is the geometry-aware entry point (a raw hit maps to exactly one primary bucket b , plus its overlap-neighbor $b-1$, per §8.1); add_to_bucket is a direct write to an already-known bucket location, used when the caller has already done bucket-location arithmetic itself (§8.4, for multi-hit seeds).

8.2.3 Propagating the seed cursor: propagate_seeds_to_buckets

Algorithm: title=propagate_seeds_to_buckets()

```
for (segm_id, b) in non_empty_buckets_with_repeats:
    buckets[segm_id][b].i := i
    buckets[segm_id][b].seeds := seeds
```

Called once, after match_seeds (§8.4) has consumed the full seed budget S for the read: every touched bucket’s own .i/.seeds counters are initialized to the *global* cursor position, so that incremental pruning (Chapter 9) can resume extending each bucket from exactly where uniform seed consumption left off, without having to replay every seed from scratch per bucket.

8.2.4 `get_sorted_buckets`

Algorithm: `title=get_sorted_buckets()`

```
sort non_empty_buckets_with_repeats by (segm_id, b)
deduplicate consecutive equal entries
result := [ (loc, buckets[loc.segm_id][loc.b]) for loc in deduplicated ]
sort result by content.matches descending
return result
```

This is the hand-off point to pruning/refinement (Chapters 9–10): buckets are visited **most-matches-first**, so that the best candidates raise the acceptance threshold as early as possible, making the seed-heuristic bound (§2.6) prune later (worse) buckets even more aggressively.

Implementation note

The reference C++ uses `std::sort`, whose relative order for *tied* match counts is unspecified (and not even guaranteed stable across compiler/standard-library versions). A from-scratch implementation using a stable sort will therefore not, in general, reproduce the exact tie-broken order the reference binary happens to produce on any given build — this is a property of the reference implementation, not a target worth chasing; downstream results only depend on tie-breaking when two buckets score *exactly* equally, which is rare in practice and does not affect which bucket is ultimately reported as long as the final selection compares scores explicitly (which it does, §10.5).

8.3 BucketsHash: ephemeral scratch structure for one multi-hit seed

A second, hash-map-backed structure with the same `add_to_pos` geometry (keyed by `BucketLoc` directly, rather than a flat per-segment array) is used *only* as short-lived scratch space, local to processing a single seed with more than one reference hit (§8.4) — never as an alternative backing store for the main per-run `Buckets`. It needs no `clear()`/undo-log machinery since a fresh instance is created per multi-hit seed and discarded immediately after.

8.4 `match_seeds`: populating buckets from the seed budget

Input. The read’s sorted seed list (Chapter 7), the (per-run, just-cleared) `Buckets`, and the seed budget S (occurrences, §7.2).

Output. None (mutates buckets in place); also returns/accumulates the total seed-match count for diagnostics.

Algorithm: `title=match_seeds(seeds, buckets, S)`

```
while buckets.i < len(seeds) and buckets.seeds < S:
    seed := seeds[buckets.i]
    if seed.hits_in_t > 0:
        if seed.hits_in_t == 1:
            hit := index.h2single[seed.kmer.h]
            content := BucketContent(matches=1, seeds=0,
                                     codirection = (hit.strand==seed.kmer.strand) ? +1 :
                                     r_min=hit.r, r_max=hit.r)
            buckets.add_to_pos(hit, content)
        else:
```

```

# De-duplicate this *one* seed's own multiple reference hits into
# per-bucket contributions *before* merging into the shared
# Buckets store, so that a seed occurring k times inside one
# bucket only ever contributes min(k, occs_in_p) matches to it
# (a seed cannot match "more than itself" in a single bucket).
scratch := BucketsHash(halfflen = buckets.halfflen)
for hit in index.h2multi[seed.kmer.h]:
    c := BucketContent(matches=1, seeds=0,
                        codirection = (hit.strand==seed.kmer.strand) ? +1 : -1,
                        r_min=hit.r, r_max=hit.r)
    scratch.add_to_pos(hit, c)
for (loc, c) in scratch.buckets:
    clamped := BucketContent(matches = min(c.matches, seed.occs_in_p), seeds=0,
                                codirection=c.codirection,
                                r_min=c.r_min, r_max=c.r_max)
    buckets.add_to_bucket(loc, clamped)
buckets.seeds += seed.occs_in_p
buckets.i += 1

```

Why the extra de-duplication pass for multi-hit seeds. A seed with, say, `occs_in_p=2` (occurs twice in the read) and `hits_in_t=5` (occurs 5 times in the reference) could have several of those 5 reference hits fall inside the *same* bucket. Without capping, that bucket would be credited with up to 5 matches from a seed that only truly occurs twice in the read — overcounting. The scratch structure first tallies, per bucket, how many of this one seed's reference hits land there, then clamps that tally to `occs_in_p` before merging into the shared Buckets store, guaranteeing a single seed can never contribute more matches to any one bucket than it actually occurs in the read.

8.5 Reference C++ implementation

```

1 void match_seeds(const Seeds &p_unique, BucketsType &B, qpos_t S) {
2     for (; B.i < (qpos_t)p_unique.size() && B.seeds < S; B.i++) {
3         Seed seed = p_unique[B.i];
4         if (seed.hits_in_T > 0) {
5             if (seed.hits_in_T == 1) {
6                 const auto &hit = tidix.h2single.at(seed.kmer.h);
7                 BucketContent content(1, 0, hit.strand == seed.kmer.strand ? 1 : -1, hit.r, hit.r);
8                 B.add_to_pos(hit, content);
9             } else {
10                 BucketsHash<abs_pos> b2m(B.halfflen);
11                 for (const auto &hit: tidix.h2multi.at(seed.kmer.h)) {
12                     BucketContent content(1, 0, hit.strand == seed.kmer.strand ? 1 : -1, hit.r, hit.r);
13                     b2m.add_to_pos(hit, content);
14                 }
15                 for (auto it = b2m.buckets.begin(); it != b2m.buckets.end(); ++it) {
16                     BucketContent content(min(it->second.matches, seed.occs_in_p), 0,
17                                             it->second.codirection, it->second.r_min, it->second.r_max);
18                     B.add_to_bucket(it->first, content);
19                 }
20             }
21         }
22         B.seeds += seed.occs_in_p;
23     }
24 }
25
26 void add_to_pos(const Hit& hit, const BucketContent &content) {
27     qpos_t b = (abs_pos ? hit.r : hit.tpos) / halfflen;
28     buckets[hit.segm_id][b] += content;
29     non_empty_buckets_with_repeats.push_back(BucketLoc(hit.segm_id, b));

```

```
30  if (b > 0) {  
31      buckets[hit.segm_id][b-1] += content;  
32      non_empty_buckets_with_repeats.push_back(BucketLoc(hit.segm_id, b-1));  
33  }  
34 }
```

Listing 8.1: `match_seeds` and `Buckets::add_to_pos`, `src/shmap.h` & `src/buckets.h`

8.6 Complexity

Populating buckets from S consumed seed occurrences costs $O(S \cdot \bar{d})$, where $\bar{d}$ is the average reference-hit count of a consumed seed (bounded by $-M$ when set, §6.2.3). `get_sorted_buckets` costs $O(u \log u)$ where u is the number of distinct buckets touched this read — typically far smaller than the reference size, since only buckets that received at least one seed match are ever touched at all.

Chapter 9

Seed-Heuristic Pruning

This chapter gives the exact implementation of the pruning bound whose theory was developed in §2.6. Three pieces compose: the bound formula itself (`hseed`), an incremental bucket-extension step (`matches_in_bucket`) that pulls in *one more* seed’s worth of information about a specific bucket on demand, and the driving loop (`seed_heuristic_pass`) that alternates “check the bound” and “extend” until the bucket is either proven prunable or fully caught up with all consumed seeds.

9.1 `hseed`: the bound formula

$$h_{\text{seed}}(m, \text{seeds}, \text{matches}) = 1 - \frac{\text{seeds} - \text{matches}}{m}$$

with m the read’s sketch size, and $\text{seeds}/\text{matches}$ the bucket’s own running counters (§4.7). See §2.6 for the proof that this is a valid, monotonically-tightening upper bound on the bucket’s final containment score.

9.2 `matches_in_bucket`: extending a bucket by one more seed

Input. The shared `Buckets` store (read-only here), a specific `BucketLoc`, that bucket’s own (mutable) running `BucketContent`, and the next seed `s` to check against it.

Output. None; mutates the passed `BucketContent` in place: increments `.seeds` by `s.occs_in_p` unconditionally, and increments `.matches` (plus updates `.codirection/.r_min/.r_max`) by however many of `s`’s reference hits fall inside this bucket’s span.

Algorithm: `title=matches_in_bucket(buckets, loc, content, s)`

```
content.seeds += s.occs_in_p
if s.hits_in_t == 0:
    return # nothing to add
if s.hits_in_t == 1:
    hit := index.h2single[s.kmer.h]
    pos := abs_pos ? hit.r : hit.tpos
    if buckets.begin(loc) <= pos < buckets.end(loc):
        content.matches += 1
        content.codirection += (hit.strand == s.kmer.strand) ? 1 : -1
        content.r_min := min(content.r_min, hit.r)
        content.r_max := max(content.r_max, hit.r)
else: # s.hits_in_t > 1
```

```

hits := index.h2multi[s.kmer.h]          # sorted by (segm_id, r); see 6.3
start := binary_search first hit with (segm_id, pos) >= (loc.segm_id, buckets.begin(lo
matches := 0
for hit in hits[start:]:
    pos := abs_pos ? hit.r : hit.tpos
    if not (hit.segm_id == loc.segm_id and pos < buckets.end(loc)):
        break          # sorted: no further hit can match either
    matches += 1
    content.codirection += (hit.strand == s.kmer.strand) ? 1 : -1
    content.r_min := min(content.r_min, hit.r)
    content.r_max := max(content.r_max, hit.r)
    content.matches += min(matches, s.occs_in_p) # same per-seed clamp as match_seeds (8.

```

The multi-hit branch’s binary search + early-break scan is exactly why `h2multi` lists are kept sorted by `(segm_id, r)` (§6.2.2): it lets this function cost $O(\log d + c)$ (binary search plus c = actual hits found in range) rather than $O(d)$ (scanning every one of the seed’s d reference occurrences), which matters because this function may be called many times per bucket during incremental pruning.

9.3 seed_heuristic_pass: the driving loop

Input. The shared `Buckets`, the read’s full seed list, the read’s sketch size m , the bucket being evaluated (location + its running content, mutable), a mutable slot to report the bound value achieved, and the current acceptance threshold `thr` (the higher of the CLI’s θ and the current best-known score, §10.5).

Output. Boolean: `true` if the bucket *survives* pruning (its bound stays $\geq$ `thr` after being extended with every seed consumed so far), `false` the instant its bound drops below `thr`.

Algorithm: `title=seed_heuristic_pass(buckets, seeds, m, loc, content, &sh, thr)`

```

if no_bucket_pruning:          # -P flag: pruning disabled entirely
    return true

loop:
    sh := hseed(m, content.seeds, content.matches)
    if sh < thr:
        return false          # proven prunable: stop immediately
    if content.i >= len(seeds):
        break                  # caught up with every consumed seed: s
    matches_in_bucket(buckets, loc, content, seeds[content.i])
    content.i += 1
return true

```

Why check the bound *before* each extension, not after. Checking first means a bucket that is already unsalvageable (e.g. its very first few seeds were all misses) is rejected in $O(1)$ — zero calls to `matches_in_bucket` — rather than paying for at least one extension step before discovering the same conclusion. Because `hseed` only ever decreases or holds steady as `content.i` advances (§2.6), checking first never risks accepting a bucket the “check after” ordering would have rejected, or vice versa — the two orderings are behaviorally equivalent except for this one-step efficiency difference.

Why `content.i` can start anywhere. Recall from §8.2.3 that every bucket’s `.i/.seeds` counters are initialized, once, to the *global* cursor value reached after `match_seeds` finished consuming the full budget S . So the very first call to `seed_heuristic_pass` for a given bucket starts its incremental extension already caught up to S (`content.i` = S ’s position in the seed list already, assuming that bucket was touched at all during `match_seeds`) — extension only ever goes *further*, past the seeds that were used to build the buckets in the first place, giving each surviving bucket a chance to be confirmed/rejected using *more* evidence than the initial seeding pass alone provided, without redoing any of that initial work.

9.4 The `no_bucket_pruning` flag (-P)

When set, `seed_heuristic_pass` degenerates to an unconditional `return true`: every bucket that survived `get_sorted_buckets` proceeds straight to full refinement (Chapter 10), with no early rejection at all. This is strictly slower and produces identical *results* to the pruned path (pruning is provably conservative, per §2.6 — it can only ever discard buckets that could not have won anyway) — its purpose is as a ground-truth oracle for testing/benchmarking the pruning logic itself, and as a fallback if a from-scratch implementation’s pruning bound is ever suspected of a bug: run both, and any discrepancy in the winning bucket indicates a pruning bug, not a scoring one.

9.5 Reference C++ implementation

```

1 double hseed(qpos_t p, qpos_t seeds, qpos_t matches) {
2     return 1.0 - double(seeds - matches) / p;
3 }
4
5 void matches_in_bucket(const BucketsType &B, const BucketLoc &b, BucketContent *bucket, const Seed &s)
6     const {
7     bucket->seeds += s.occs_in_p;
8     if (s.hits_in_T == 0) {
9         ;
10    } else if (s.hits_in_T == 1) {
11        const auto &hit = tidx.h2single.at(s.kmer.h);
12        if (abs_pos ? B.begin(b) <= hit.r && hit.r < B.end(b)
13            : B.begin(b) <= hit.tpos && hit.tpos < B.end(b)) {
14            bucket->matches += 1;
15            bucket->codirection += hit.strand == s.kmer.strand ? 1 : -1;
16            bucket->r_min = min(bucket->r_min, hit.r);
17            bucket->r_max = max(bucket->r_max, hit.r);
18        }
19    } else if (s.hits_in_T > 1) {
20        const auto &hits = tidx.h2multi.at(s.kmer.h);
21        auto it = lower_bound(hits.begin(), hits.end(), B.begin(b), [&b](const Hit &hit, rpos_t pos) {
22            if (hit.segm_id != b.segm_id) return hit.segm_id < b.segm_id;
23            return abs_pos ? hit.r < pos : hit.tpos < pos;
24        });
25        rpos_t matches = 0;
26        for (; it != hits.end(); ++it) {
27            if (abs_pos) { if (!(it->segm_id == b.segm_id && it->r < B.end(b))) break; }
28            else { if (!(it->segm_id == b.segm_id && it->tpos < B.end(b))) break; }
29            matches += 1;
30            bucket->codirection += it->strand == s.kmer.strand ? 1 : -1;
31            bucket->r_min = min(bucket->r_min, it->r);
32            bucket->r_max = max(bucket->r_max, it->r);
33        }
34        bucket->matches += min(matches, s.occs_in_p);
35    }
36 }
37
38 bool seed_heuristic_pass(const BucketsType &B, const Seeds &p_unique, qpos_t m, const BucketLoc &b,
39     BucketContent *bucket, double *sh, double thr) {
40     if (!no_bucket_pruning) {

```

```

40     while (1) {
41         *sh = hseed(m, bucket->seeds, bucket->matches);
42         if (*sh < thr) return false;
43         if (bucket->i >= (qpos_t)p_unique.size()) break;
44         matches_in_bucket(B, b, bucket, p_unique[bucket->i]);
45         bucket->i++;
46     }
47 }
48 return true;
49 }

```

Listing 9.1: hseed, matches_in_bucket, seed_heuristic_pass, src/shmap.h

9.6 Worked example

Suppose $m = 100$ (read sketch size), $thr = 0.9$, and a candidate bucket's running counters after seeding are $seeds = 40$, $matches = 38$. Then $h_{seed} = 1 - (40 - 38)/100 = 0.98 \geq 0.9$: survives, extend further. Suppose the next three consumed seeds (each $occs_in_p = 1$) contribute 0, 1, 0 matches to this bucket respectively: after seed 1, $seeds = 41$, $matches = 38 \Rightarrow h_{seed} = 0.97$; after seed 2, $seeds = 42$, $matches = 39 \Rightarrow h_{seed} = 0.97$; after seed 3, $seeds = 43$, $matches = 39 \Rightarrow h_{seed} = 0.96$ — still above 0.9, so this bucket keeps surviving each check. If instead six consecutive misses had occurred ($seeds$ climbing to 46, $matches$ frozen at 38), $h_{seed} = 1 - 8/100 = 0.92$, still surviving; a seventh miss would give $1 - 9/100 = 0.91$, an eighth 0.90 (still $\geq$, survives), a ninth $0.89 < 0.9$: **pruned** at that point, having spent only 9 incremental extension calls to conclusively rule out a bucket that (if fully scored) might have needed to examine every remaining position in its span.

Chapter 10

Refinement and Scoring

Buckets that survive pruning (Chapter 9) are handed to *refinement*: compute an exact similarity score, and the precise matched sub-interval, for the bucket. This chapter covers all four scoring metrics (§3.3), the two-pointer sliding-window sweep that underlies `Containment/Jaccard` scoring, the LCS-based alternative, and `match_rest`, the procedure that ties refinement into a best/second-best search over all of a read’s surviving buckets.

10.1 The `diff_hist` bookkeeping device

Every sliding-window sweep in this chapter maintains a shared per-read map `diff_hist: hash → remaining budget`, initialized once per read from each seed’s `occs_in_p` (how many times that k -mer occurs in the read). As the sweep’s right (or left) boundary crosses a reference position whose k -mer hash is a read seed, the corresponding budget entry is decremented (window grows) or incremented (window shrinks) by exactly one, and the *sign* of the result after a decrement determines whether that occurrence is countable as part of the intersection:

- Decrementing (window grows, position enters): if the value *after* decrementing is still ≥ 0 , this occurrence is a legitimate, not-yet-used match — count it, and record that it was counted (so the corresponding increment, when the window later shrinks past it, knows to un-count it).
- Incrementing (window shrinks, position leaves): if the value *after* incrementing is ≥ 1 , this position had previously been counted — un-count it.

This lets a sliding window maintain an exact intersection-with-multiplicity count in $O(1)$ amortized per boundary move, correctly handling repeated k -mers (a hash occurring 3 times in the read can be “used up” by at most 3 reference positions inside the current window, no matter how many more copies of that hash the reference bucket actually contains) — this is the *multiset* intersection a containment/Jaccard estimate needs (§2.3.3), not a plain set intersection.

Implementation note

Because a window can shrink and regrow over the same reference position multiple times as the sweep’s left boundary advances, `diff_hist` values legitimately go negative *transiently* (a decrement can push a shared hash’s budget below what’s currently “available” if multiple reference positions inside the window share that hash but the read only has fewer occurrences) — this is expected and exactly what the ≥ 0 / ≥ 1 sign checks are designed to handle correctly; do not add an assertion that rejects negative values.

10.2 SHMapper's own bestFixedLength: the main refinement sweep

This is the function that actually refines Containment/Jaccard buckets in the main mapping path (there is a second, differently-shaped function with the same name used only by the ground-truth diagnostic path, §10.6 — keep them distinct).

Input. The reference segment (its full sketch), a window [from, to) to sweep (the bucket's own span, from buckets.begin/end), the read's seed lookup table (hash → Seed), diff_hist (mutable, shared across the whole read's refinement calls), the read length minus k (used as the PAF end-coordinate, not a window bound), the window *width* bound m (confusingly, at the call site this is actually the read's containing $\ell/l_{\max}$ value, not the sketch size — see the note below), and the metric (Containment or Jaccard).

Output. A Mapping (§4.10): the best-scoring sub-window found anywhere inside [from, to).

Algorithm: title=bestFixedLength(segm, from, to, seed_table, diff_hist, p_sz, m, metric)

```

t := segm.kmers
l := max(from, 0)
r := l
end := min(len(t), to)
best := Mapping::default()           # score = -1, beaten by any real candidate

while l < end:
    while r < end:
        # window-width check, in whichever coordinate space abs_pos selects
        if abs_pos: if not (t[r].r < t[l].r + m): break
        else:      if not (r < l + m): break
        if t[r].h in seed_table:
            same_strand += codirection(seed_table[t[r].h].kmer, t[r])
            if (--diff_hist[t[r].h]) >= 0:
                intersection += 1
        r += 1
    # r is now exactly one past the last position included in this window
    # for the CURRENT l -- this is the point at which it is safe to score,
    # since r has already been advanced past every included element.
    s_kmers := r - l
    score := mappingScore(intersection, m, s_kmers, metric)           # 10.2 below
    if l < r and score > best.score():
        best.update(p_start=0, p_end=p_sz-1, t_l=t[l].r, t_r=t[r-1].r,
                    segm=segm, intersection=intersection, score=score,
                    same_strand=same_strand, span=t[r-1].r - t[l].r + 1)
    if t[l].h in seed_table:
        same_strand -= codirection(seed_table[t[l].h].kmer, t[l])
        if (++diff_hist[t[l].h]) >= 1:
            intersection -= 1
    l += 1
return best

```

mappingScore:

$$\text{mappingScore}(\text{intersection}, m, s_{\text{sz}}, \text{Jaccard}) = \frac{\text{intersection}}{m + s_{\text{sz}} - \text{intersection}}, \quad \text{mappingScore}(\text{intersection}, m, s_{\text{sz}}, \text{Containment}) = \frac{\text{intersection}}{m}$$

These are exactly the FracMinHash Jaccard/containment estimators of §2.3.3, with m the read’s sketch size and s_sz the current window’s k -mer count.

Implementation note

At the actual call site (§10.3), this function’s `p_sz` parameter receives *read length minus k* , and its `m` parameter receives *$lmax$* (the bucket half-length ℓ), *not* the read’s sketch size in either case, despite parameter names that suggest otherwise at a casual read. This is not a bug — ℓ was chosen (§8.1.1) to equal the read’s sketch size in the default (non-abs_pos) configuration, so the values coincide numerically for the common case, but a from-scratch implementation should pass the value that is actually semantically the “window-width bound and Jaccard/containment denominator” at this call site, not necessarily whatever local variable happens to be named `m` in a hasty transcription. Name the parameter for what it does, not what one caller happens to pass.

Why scoring happens *after* the inner while loop exits, not inside it. This is the single most important structural detail in this function, and the source of a real bug in the ground-truth-only sibling function discussed in §10.6: after the inner loop’s normal exit, `r` is *exactly one past* the last position that was ever included in the window for the current `l` (standard half-open-range convention: the loop only stops once its own condition fails, having already processed and advanced past every element that satisfied it). So `t[r-1]` unambiguously means “the last element actually in the window”, consistent with what *intersection* counts. Any refinement sweep that instead scores *mid-loop* (before the equivalent exit has happened) must use the *current*, not-yet-advanced index for both the score’s span *and* its use of *intersection*, or the two will disagree — see §10.6 for a concrete case where the reference implementation gets exactly this wrong.

10.3 findBestMapping: dispatching on metric

Algorithm: `title=findBestMapping(buckets, loc, content, seed_table, diff_hist, p_sz, m, lmax, sh, metric, k)`

```
switch metric:
  case bucket_SH:
    # Degenerate/cheap: reuse the seed-heuristic bound itself as the score,
    # with the bucket’s own tracked position range as the matched interval.
    # No refinement sweep at all -- O(1) given the bucket’s content.
    best := Mapping(p_start=0, p_end=p_sz-1, t_l=content.r_min, t_r=content.r_max,
                    segm=ref_segment(loc.segm_id), intersection=content.matches,
                    score=sh, same_strand=content.codirection,
                    span=content.r_max - content.r_min)

  case bucket_LCS:
    lcs_len := lcs(ref_segment(loc.segm_id).kmers, buckets, loc, seed_table) # 10.4
    score := lcs_len / m
    best := Mapping(p_start=0, p_end=p_sz-1, t_l=content.r_min, t_r=content.r_max,
                    segm=ref_segment(loc.segm_id), intersection=content.matches,
                    score=score, same_strand=content.codirection,
                    span=content.r_max - content.r_min)

  case Containment or Jaccard:
```

```

    best := bestFixedLength(ref_segment(loc.segm_id), buckets.begin(loc), buckets.end(loc),
                           seed_table, diff_hist, p_sz - k, lmax, metric)

    best.set_bucket(loc)
    best.set_sh(sh)
    return best

```

10.4 The `bucket_LCS` metric: longest common subsequence chain score

Motivation. Containment/Jaccard scoring treats every matched k -mer equally, regardless of whether the matches are mutually *co-linear* (consistent with one real alignment) or scattered (an artifact of repeats). `bucket_LCS` instead scores a bucket by how long a co-linear chain of matches it contains, per §2.5.

10.4.1 `lcs_get_ppos_in_t`: collecting the position sequence

Algorithm: `title=lcs_get_ppos_in_t(t, buckets, loc, seed_table)`

```

ppos_in_t := []
for l in buckets.begin(loc) .. min(len(t), buckets.end(loc)) - 1:
    if t[l].h in seed_table:
        for ppos in seed_table[t[l].h].pmatches:      # Section 4.4: all read positions sh
            ppos_in_t.append(ppos)
return ppos_in_t

```

This produces, in increasing reference-position order, the sequence of read-side positions that each reference k -mer inside the bucket matches (with repetition, if a hash occurs multiple times in the read). A long increasing run in this sequence is a long co-linear chain.

10.4.2 `lcs_get_lis`: patience-sorting LIS length

Exactly the classic $O(n \log n)$ algorithm of §2.5:

Algorithm: `title=lcs_get_lis(ppos_in_t)`

```

tails := []                                # tails[i] = smallest tail of an increasing run of length i
for x in ppos_in_t:
    lo, hi := -1, len(tails)
    while lo + 1 < hi:
        mid := (lo + hi) / 2
        if tails[mid] < x: lo := mid
        else: hi := mid
    if hi == len(tails): tails.append(x)
    else: tails[hi] := x
return tails                                # len(tails) is the LIS length

```

10.4.3 lcs: combining forward and reverse

Algorithm: `title=lcs(t, buckets, loc, seed_table)`

```
ppos_in_t := lcs_get_ppos_in_t(t, buckets, loc, seed_table)
forward_len := len(lcs_get_lis(ppos_in_t))
reverse(ppos_in_t)
reverse_len := len(lcs_get_lis(ppos_in_t))
return max(forward_len, reverse_len)
```

Running LIS on the *reversed* position sequence finds the longest *decreasing* run of the original — i.e., the longest chain co-linear on the opposite strand. Taking the max of both directions makes this metric strand-agnostic without needing to know the winning strand ahead of time.

10.5 match_rest: best and second-best search

Input. All of a read’s sorted, surviving buckets; the read’s seed list, seed table, and `diff_hist`; an acceptance threshold `thr`; an optional “forbidden” Mapping (when searching for the second-best, this is the best mapping already found, so that a near-duplicate of it is excluded, §10.5.1); and `max_overlap` (CLI `-o`).

Output. The single best Mapping across all surviving buckets that both beats `thr` and (if `forbidden` is given) does not overlap it by more than `max_overlap`; or none, if no bucket qualifies.

Algorithm: `title=match_rest(p_sz, m, lmax, seeds, buckets, sorted_buckets, diff_hist, seed_table, thr, forbidden, max_overlap, metric, k)`

```
best := None
for (loc, content) in sorted_buckets:                # most-matches-first, Section 8.6
    sh := 1.0
    if seed_heuristic_pass(buckets, seeds, m, loc, content, &sh, thr): # Chapter 9
        candidate := findBestMapping(buckets, loc, content, seed_table, diff_hist,
                                     p_sz, m, lmax, sh, metric, k)
        if candidate.score() > thr:
            if forbidden is None or overlap(candidate, forbidden) < max_overlap:
                thr := candidate.score()           # raise the bar for subsequent buckets
                best := candidate
return best
```

10.5.1 Overlap exclusion

$$\text{overlap}(a, b) = \begin{cases} -0.0 & \text{different segments} \\ \frac{\max(0, \min(a.t_r, b.t_r) - \max(a.t_l, b.t_l))}{\max(a.t_r, b.t_r) - \min(a.t_l, b.t_l)} & \text{same segment} \end{cases}$$

i.e., intersection-over-union of the two candidates’ matched reference intervals, restricted to same-segment pairs. A second-best candidate search rejects any bucket whose refined interval overlaps the already-found best by $\geq \text{max_overlap}$ — without this, the “second best” would routinely just be a slightly-shifted restatement of the exact same true alignment (since neighboring, heavily-overlapping buckets, per §8.1, often all score similarly well against the same true locus), which would make MAPQ (Chapter 12) uselessly report low confidence for essentially every uniquely-mapping read.

10.5.2 How the caller invokes `match_rest` twice

The per-read driver (fully assembled in Chapter 11) calls `match_rest` exactly twice: once with `thr =  $\theta$` (the CLI acceptance threshold) and `forbidden=None` to find the best mapping; then, only if a best mapping was found, a second time with `thr =  $best.score() \times (1 - min\_diff)$` and `forbidden=best` to find the second-best. Both calls sweep the *same* sorted bucket list and share the *same* `diff_hist` (which is why every refinement sweep is required to leave `diff_hist` in exactly the state it found it, §10.1 — the second call depends on that invariant holding after the first).

10.6 The ground-truth-only sibling functions, and a genuine bug

A second, narrower set of functions (grouped under a type usually called `Matcher`) exists solely to support the *verbose ground-truth diagnostic* output (§13.3, used only with `-v 2`, never in the main scoring path above). It re-implements `collect_matches` (turn a bucket into an explicit list of `Matches`, rather than sweeping the reference sketch directly), a `bestFixedLength` that sweeps that explicit list using `hit.r <= l.hit.r + p.sz` as its window bound (notably *not* respecting an `lmax` parameter it is nonetheless passed — see the dead-parameter note in Chapter 16), and a `bestIncludedJaccard` that finds the best-scoring window whose length falls within an explicit $[\ell_{min}, \ell_{max}]$ range (used to check, precisely, what score the ground-truth region itself would have achieved).

Known defect in the reference implementation

`bestIncludedJaccard`’s two-pointer sweep scores *mid-loop*, immediately after growing the window’s right boundary by one, using the position *one before* the just-processed element (an iterator idiom equivalent to `prev(r)`) for both the matched span and the reported right boundary — while `intersection` at that exact point *already* reflects the just-processed (current, not “previous”) element. This is an internal inconsistency: the score’s span excludes an element that the match count includes, which can and does produce `intersection > span`, violating the metric’s own well-formedness invariant (*intersection* $\leq$ *s.sz* must hold for a containment/Jaccard ratio to stay in $[0, 1]$). The original source itself carries a `// TODO: should it be prev(r) instead?` comment on this exact line, i.e. the original author flagged this specific spot as unsettled. Confirmed by constructing a small worked example (a ~ 50 bp read against a ~ 200 bp synthetic reference) that reliably triggers the invariant violation. Because this path has *zero* automated test coverage in the reference codebase and only ever affects a diagnostic tag (never the main PAF output), there is no existing oracle to defer to; the internally consistent fix — score using the *current*, just-processed right-boundary element throughout, matching what `intersection` already counts, exactly as §10.2’s “score after the loop exits” pattern already does correctly — is the recommended behavior for a from-scratch implementation of this diagnostic feature. See Chapter 16 for the full write-up.

10.7 Complexity

Each refinement sweep (`bestFixedLength` or `lcs`) costs $O(|\text{bucket}|)$: one linear pass over the bucket’s span of the reference sketch, with $O(1)$ amortized work per position via the `diff_hist` device (§10.1) or, for LCS, $O(|\text{bucket}| \log |\text{bucket}|)$ owing to the patience-sorting binary search. Summed over all buckets that survive pruning (Chapter 9), and since pruning is designed to make that survivor count small and roughly independent of total reference size, total refinement cost per read is, in practice, far closer to $O(\ell)$ (one bucket’s worth of work) than to $O(n)$ (the whole reference).

Chapter 11

End-to-End Algorithm

This chapter reassembles every module from Chapters 5–10 into the two top-level routines that actually run: `map_reads` (once per program invocation) and `map_read` (once per read). This is the single most useful reference for a from-scratch implementation, and is written in its *corrected* form (i.e., incorporating the fixes from Chapter 16 where they affect observable behavior) with explicit call-outs at every point a decision was made about a defect in the reference source.

11.1 `map_reads`: the outer loop

Algorithm: `title=map_reads(index, reads_file, params)`

```
open unmapped_out  := params.params_file + ".unmapped.paf"  if params.params_file is set e
open paulout       := params.params_file + ".paul.tsv"      if params.params_file is set
                                                           and params.verbose >= 2 el

buckets := Buckets::new(index)          # allocated once, cleared per read (Section 8.2
run_wide_counters := Counters::new()
run_wide_timers   := Timers::new()

for (query_id, read_seq) in read_fasta(reads_file):
    run_wide_counters.inc("reads")
    per_read_counters, per_read_timers := map_read(index, params, read_seq, query_id,
                                                    buckets, unmapped_out, paulout)

    run_wide_counters += per_read_counters      # accumulate into run totals
    run_wide_timers   += per_read_timers

print_summary_statistics(run_wide_counters, run_wide_timers)
```

Implementation note

Each read is processed with its own fresh, fully-reset per-read counters (see the boxed correction in §11.2.1 below) that are merged into the run-wide totals *after* that read finishes — this merge-after-each-read pattern is what lets a single PAF line’s diagnostic tags (§13) report purely this-read quantities while the end-of-run summary still reports correct totals across all reads.

11.2 map_read: per-read driver

11.2.1 Counter initialization

Known defect in the reference implementation

The reference C++ does *not* actually reset its per-read counters object between reads: the call that would clear it (`C.clear()`) is commented out, and the subsequent re-initialization list names only a subset of the counters the function goes on to increment — seven counters, including `total_matches` (which feeds directly into the *live* `total_matches:i:/match_inefficiency:f`: PAF tags, not just an end-of-run summary), are left to accumulate, unreset, across every read a single mapper instance processes, compounding further every time the (also unreset) running total is merged into the run-wide counters after each read. The correct, obviously-intended behavior — and the one specified below — is to **fully reset all per-read counters at the start of every read**, with every counter name the function goes on to touch pre-registered at zero (so that end-of-read diagnostic reads never fail on a genuinely-never-incremented counter, e.g. `mapq60` for a read with a low-confidence mapping). See Chapter 16 for the full analysis and the complete list of affected counter names.

Algorithm: title=map_read — part 1: sketch, seed, prepare

```

C := Counters::new()                # fresh, fully-reset every read (see box above)
T := Timers::new()
buckets.clear()

p := sketcher.sketch(read_seq)      # Chapter 5
m := len(p)                         # read's sketch size
C.inc("read_len", len(read_seq))

p_unique := unique_elements_with_info(p) # Chapter 7; rarest-hit-first

seed_table := {}                   # hash -> Seed
diff_hist := {}                   # hash -> remaining occurrence budget
possible_matches := 0
for seed in p_unique:
    seed_table[seed.kmer.h] := seed
    diff_hist[seed.kmer.h] := seed.occs_in_p
    possible_matches += seed.hits_in_t
C.inc("possible_matches", possible_matches)

lmax := m
theta := params.theta
theta2 := theta - params.min_diff
S := one_sweep ? len(p_unique)      # best-effort -B interpretation, Ch.
    : floor((1 - theta2) * m) + 1    # Section 7.2

if abs_pos: buckets.set_halflen(len(read_seq))
else:        buckets.set_halflen(m)
if buckets.halflen < MIN_HALFLEN:
    goto unmapped                    # read too short to bucket meaningf

```

Algorithm: title=map_read — part 2: match seeds, prune, refine

```

match_seeds(p_unique, buckets, S)                                # Chapter 8
buckets.propagate_seeds_to_buckets()
sorted_buckets := buckets.get_sorted_buckets()                  # most-matches-first
C.inc("seeded_buckets", len(sorted_buckets))

best := match_rest(len(read_seq), m, lmax, p_unique, buckets, sorted_buckets, diff_hist,
                  seed_table, theta, forbidden=None, params.max_overlap, params.metric,

best2 := None
if best is not None:
    second_best_thr := best.score() * (1 - params.min_diff)
    best2 := match_rest(len(read_seq), m, lmax, p_unique, buckets, sorted_buckets, diff_hist,
                      seed_table, second_best_thr, forbidden=best, params.max_overlap,
                      params.metric, params.k)

```

11.2.2 Output: mapped vs. unmapped**Known defect in the reference implementation**

The reference C++ calls `best->set_global_stats(...)` and `best->print_paf(...)` **unconditionally**, even when `best` is empty (every unmapped read) — dereferencing an empty `std::optional`, undefined behavior in C++ (and simply un-representable against a proper `optional/Option` type in a memory-safe language, since there is no value to dereference). Per the decision recorded for this document’s companion re-implementation, unmapped reads should instead get a well-defined, minimal record: write `query_id`, read length, and `*/zero` placeholder fields for the alignment-specific columns to the “unmapped” output stream (only opened at all if a `params-file/output` prefix was configured, matching the reference’s own gating for that stream). See Chapter 16 for the full discussion of why this specific choice (rather than, say, silently skipping unmapped reads entirely) was made.

Algorithm: title=map_read — part 3: emit result

```

if best is not None:
    if best2 is not None:
        best.set_second_best(best2)                                # Chapter 12: feeds MAPQ
    C.inc("matches_in_reported_mappings", best.intersection())
    C.inc("J_best", round(10000 * best.score()))
    C.inc("mappings"); C.inc("mapped_reads")

    best.set_global_stats(theta2, params.min_diff, sketch_size=m, query_id, read_len=len(r),
                          params.k, seeds=S, C.total_matches, C.max_seed_matches, C.seed_
                          C.seeded_buckets, C.final_buckets, fftp=-1.0, map_time=T.secs("
                          # ^ calc_mapq() runs inside set_global_stats; Chapter 12

    emit_paf_line(stdout, best)                                    # Chapter 13
    if best.mapq() == 60: C.inc("mapq60")
    if best.mapq() == 0: C.inc("mapq0")

    if params.verbose >= 2:
        gt := AnalyseSimulatedReads(query_id, read_seq, diff_hist, m, seed_table, index,
                                     buckets, params.theta)                                # Section 13.x

```



```

    emit_paf_ground_truth_tags(stdout, gt, best)
    if paulout is not None: gt.print_tsv(paulout)
else:
    emit_minimal_unmapped_record(unmapped_out, query_id, len(read_seq)) # the fix above
    if params.verbose >= 2 and paulout is not None:
        gt := AnalyseSimulatedReads(query_id, read_seq, diff_hist, m, seed_table, index,
                                   buckets, params.theta)

        gt.print_tsv(paulout)

return C, T

```

11.3 Full call graph

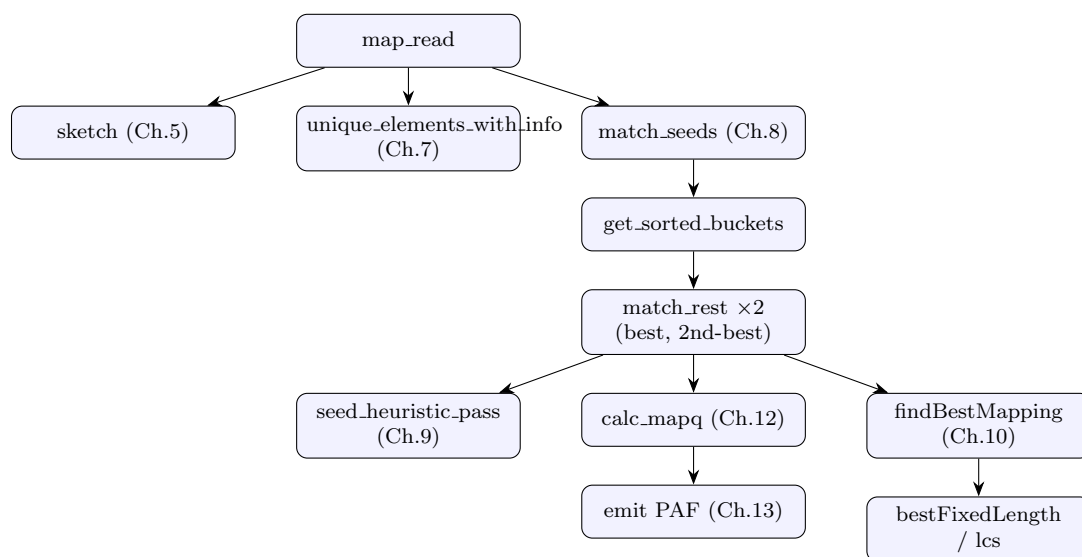


Figure 11.1: Call graph of one `map_read` invocation.

11.4 Reference C++ implementation (annotated)

```

1 inline void map_read(const params_t &params, const FracMinHash &sketcher,
2                     ofstream &paulout, ofstream &unmapped_out,
3                     const string &query_id, const string &P, BucketsType &B) {
4     T.clear();
5     B.clear();
6     // NOTE: see Chapter 16 -- C.clear() is missing here in the reference source.
7
8     sketch_t p = sketcher.sketch(P);
9     qpos_t m = p.size();
10    C.inc("read_len", P.size());
11
12    Seeds p_unique = unique_elements_with_info(p);
13
14    h2seed_t p_ht; h2cnt diff_hist; rpos_t possible_matches(0);
15    for (const auto &kmer: p_unique) {
16        p_ht.insert(make_pair(kmer.kmer.h, kmer));
17        diff_hist[kmer.kmer.h] = kmer.occs_in_p;
18        possible_matches += kmer.hits_in_T;
19    }
20
21    auto lmax = m;
22    auto theta = params.theta;

```

```

23 double theta2 = theta - params.min_diff;
24 qpos_t S = qpos_t((1.0 - theta2) * m) + 1;
25
26 if (abs_pos) B.set_halflen(P.size()); else B.set_halflen(m);
27
28 match_seeds(p_unique, B, S);
29 B.propagate_seeds_to_buckets();
30 auto sorted_buckets = B.get_sorted_buckets();
31
32 std::optional<Mapping> best, best2;
33 best = match_rest(P.size(), m, lmax, p_unique, B, sorted_buckets, diff_hist, p_ht,
34                 theta, std::nullopt, query_id, &lost_on_pruning, params.max_overlap);
35 if (best) {
36     double second_best_thr = best->score() * (1.0 - params.min_diff);
37     best2 = match_rest(P.size(), m, lmax, p_unique, B, sorted_buckets, diff_hist, p_ht,
38                     second_best_thr, best, query_id, &lost_on_pruning, params.max_overlap);
39 }
40
41 if (best) {
42     if (best2) best->set_second_best(best2.value());
43     os = &cout;
44 } else {
45     os = &unmapped_out;
46 }
47 // NOTE: see Chapter 16 -- the following two calls are unconditional in the reference
48 // source even when 'best' is empty; this listing shows the corrected, guarded form.
49 best->set_global_stats(theta2, params.min_diff, m, query_id.c_str(), P.size(), params.k,
50                     S, C.count("total_matches"), C.count("max_seed_matches"),
51                     C.count("seed_matches"), C.count("seeded_buckets"),
52                     C.count("final_buckets"), -1.0, T.secs("query_mapping"));
53 best->print_paf(*os);
54 }

```

Listing 11.1: `map_read, src/shmap.h` — core structure (elided: verbose-mode/diagnostic branches, identical in spirit to the pseudocode above)

Chapter 12

Mapping Quality Estimation

12.1 Inputs

By the time MAPQ is computed, a read has: a best `Mapping` with score $f_1 \in [0, 1]$ and signed same-strand-seed count σ_1 ; optionally a second-best `Mapping` with score $f_2 \leq f_1$ (via `set_second_best`, below); and the run parameters θ_2 (best-minus- `min_diff` threshold, §11.2) and `min_diff` (CLI `-d`).

12.2 `set_second_best`

Algorithm: `title=Mapping::set_second_best(m2)`

```
self.bucket2      := m2.bucket
self.intersection2 := m2.intersection
self.score2       := m2.score()
self.sigmas_diff  := sigmas_diff(self.intersection2, self.intersection)

function sigmas_diff(X, Y):
    X := (X == -1) ? 0 : X           # sentinel "no second best" reads as 0
    Y := (Y == -1) ? 0 : Y
    return abs(X - Y) / sqrt(X + Y)
```

`sigmas_diff` is a diagnostic-only quantity (emitted as the `Isdiff:f`: PAF tag, §13): treating the intersection counts as approximately Poisson-distributed, $|X - Y|/\sqrt{X + Y}$ is the familiar “number of standard deviations apart” statistic for a difference of two counts with variance $\approx$ their sum — a quick, cheap significance proxy for how distinguishable the best and second-best candidates’ match counts are. It does not feed into `calc_mapq` itself.

12.3 `calc_mapq`: the concrete formula

Algorithm: `title=Mapping::calc_mapq( $\theta_2$ , min_diff)`

```
if score2 < theta2:
    score2 := theta2                # clamp: treat "no real 2nd best" as if it were
frac := 1 - score2 / score()
# integer division on purpose -- see note below
mapq_strand := (abs(same_strand_seeds) < intersection / 2) ? 5 : 60
if frac > min_diff:
```

```

    return min(60, mapq_strand)
else:
    return 0

```

Reading the formula. Two independent signals are combined:

1. **Score separation:** $frac = 1 - f_2/f_1$ (using the second-best score, or θ_2 if there was no real second-best candidate or it scored below θ_2). Large $frac$ means the best candidate is far ahead of any rival — confident. The threshold for “confident enough” is exactly `min_diff` (CLI `-d`): the same parameter that also sizes the seed budget (§7.2) and the second-best search’s own acceptance threshold (§10.5) is reused here as the MAPQ confidence cutoff too.
2. **Strand consistency:** `same_strand_seeds` is the best mapping’s own signed codirection tally (§4.5); if fewer than half the matched k -mers agree with the reported strand (i.e., the mapping’s own internal evidence for its stated orientation is weak), the achievable MAPQ is capped at 5 rather than 60, regardless of how good the score separation is.

If $frac > min_diff$: report $\min(60, mapq_strand)$ — i.e., either 60 (confident, strand-consistent) or 5 (confident on score alone, but strand-ambiguous). If $frac \leq min_diff$: report 0 unconditionally (not confident) — notice this is an **all-or-nothing** step function, not a smooth gradient between 0 and 60.

Implementation note

The integer division `intersection / 2` in the strand-consistency check is exactly that — integer, truncating division, since `intersection` is stored as a 32-bit integer type in the reference implementation. A from-scratch implementation that instead computes this comparison using floating-point division will disagree with the reference for odd-valued `intersection` counts (e.g. `intersection=5`: integer division gives $5/2 = 2$, so `same_strand_seeds = 2` evaluates as *not* < 2 , hence `mapq_strand = 60`; floating-point division gives 2.5, and $2 < 2.5$ evaluates true, hence `mapq_strand = 5` — a different MAPQ result for the identical input). Preserve integer truncation here.

12.4 Two commented-out alternative formulas

The reference source retains, directly above the live formula and clearly disabled (C++ `//` comments), two alternative MAPQ schemes:

1. A ratio-based scheme using $frac = (1 - f_2)/(1 - f_1)$ with three tiers (full confidence above a threshold, zero below half that threshold, linear interpolation in between) — a smoothly graduated version of the same idea, closer in spirit to minimap2’s own published MAPQ formula ($40(1 - f_2/f_1) \min(1, m/10) \log f_1$, referenced in a comment directly above these alternatives).
2. A difference-based scheme using $diff = f_1 - f_2$ directly (rather than a ratio), with the same three-tier linear-interpolation structure.

Their presence, never deleted, alongside the currently-live all-or-nothing formula, is evidence of active, unfinished tuning rather than a settled design; a from-scratch implementation aiming purely to reproduce shmap’s *shipped* behavior should implement only the live formula given in §12.3 above, but an implementation aiming to *improve* on it might reasonably start from these alternatives (or from minimap2’s own published formula) instead — this is flagged as a design choice, not a defect, since no test or shipped behavior depends on which of the three is used going forward.

12.5 Reference C++ implementation

```
1 int calc_mapq(double theta2, double min_diff) {  
2     if (score2() < theta2) set_score2(theta2);  
3     double frac = 1.0 - score2()/score();  
4     int mapq_strand = (abs(local_stats.same_strand_seeds) < local_stats.intersection/2) ? 5 : 60;  
5     if (frac > min_diff) {  
6         return std::min(60, mapq_strand);  
7     } else {  
8         return 0;  
9     }  
10 }
```

Listing 12.1: `Mapping::calc_mapq`, `src/sketch.h` (live formula; the two commented-out alternatives immediately precede it in the source and are omitted here)

12.6 When is `calc_mapq` actually invoked

Exactly once per *mapped* read, from inside `Mapping::set_global_stats` (§11.2.2), *after* `set_second_best` has already been called if a second-best candidate exists (so that `score2()` reflects it) — if no second-best was found at all, `score2()` remains at its default sentinel (-1.0), which the clamp `if (score2() < theta2) set_score2(theta2)` converts into exactly θ_2 before the ratio is computed, giving a well-defined, maximally-confident result for uniquely-mapping reads without any special-casing needed in the formula itself.

Chapter 13

Output Format

shmap emits one line per *mapped* read to standard output, in an extended PAF format: 12 mandatory tab-separated PAF columns, followed by a long tail of `name:type:value` optional tags in the SAM-style convention (`type` one of `i` integer, `f` float, `s` string). Unmapped reads are written, one per line, to a separate `.unmapped.paf` file (only created at all if an output prefix was configured, CLI `-z`).

13.1 The 12 mandatory PAF columns

#	Column	Source field	Meaning
1	query name	<code>query_id</code>	Read id (FASTA header, truncated at first whitespace).
2	query length	<code>p_sz</code>	Read length in nucleotides.
3	query start	<code>p_start</code>	Always 0 (shmap does not clip/trim the query side).
4	query end	<code>p_end</code>	Always <code>p_sz-1</code> under the current implementation — see the note below; effectively always the full read length.
5	strand	<code>strand</code>	+ if the mapping's <code>same_strand_seeds</code> tally is positive, - otherwise.
6	target name	<code>segm_name</code>	Reference segment name (FASTA header).
7	target length	<code>segm_sz</code>	Reference segment length in nucleotides.
8	target start	<code>t_l</code>	Leftmost matched reference nucleotide position.
9	target end	<code>t_r</code>	Rightmost matched reference nucleotide position.
10	# matches	<code>p_sz (!)</code>	See defect note below — this column is supposed to be the actual residue-match count but instead always repeats the query-length column.
11	block length	<code>p_sz (!)</code>	Same defect: supposed to be the alignment block length, instead repeats query length.
12	mapping quality	<code>mapq</code>	Chapter 12's output, {0,5,60} (or 255 for “not yet computed”, never actually observed in emitted output).

Known defect in the reference implementation

Columns 10 and 11 (“number of residue matches” and “alignment block length” in the PAF specification) are, in the reference source, both hardcoded to repeat column 2 (query length) rather than reporting the mapper’s own computed intersection/span — marked with a `// TODO: fix` comment in the source itself. A from-scratch implementation aiming for a *spec-conformant* PAF file (e.g. for compatibility with downstream tools that read these columns meaningfully, such as coverage calculators) should populate column 10 with the mapping’s `intersection` count and column 11 with its matched span (`t.r - t.l + 1`); one aiming purely to reproduce shmap’s own shipped output byte-for-byte should keep the repeated-query-length placeholder.

13.2 Custom diagnostic tags (mapped reads)

Appended, in this exact order, after the 12 mandatory columns. All tags below are always present on every mapped-read output line (none are conditional).

13.2.1 Per-candidate tags (LocalMappingStats)

Tag	Type	Meaning
p:i:	int	Sketch size (m) considered for this read.
s:i:	int	Matched reference span (nucleotides or sketch-index units, per <code>abs_pos</code>).
I:i:	int	Best mapping's intersection (matched k -mer) count.
I2:i:	int	Second-best mapping's intersection count (-1 sentinel if none).
Idiff:i:	int	$I - I2$ (note: if $I2$ is the -1 sentinel, this is $I - (-1) = I + 1$, <i>not</i> I , since the raw sentinel value — not the “treat as 0” convention used by <code>sigmas_diff</code> , §12 — is used directly here).
Isdiff:f:	float	<code>sigmas_diff(I2, I)</code> (§12; here the $-1 \rightarrow 0$ convention <i>does</i> apply, since it's computed by that function).
J:f:	float	Best mapping's score (whichever metric was selected, CLI <code>-m</code>).
J2:f:	float	Second-best score (-1 sentinel if none, <i>not</i> clamped to θ_2 here — that clamping is a side effect specific to <code>calc_mapq</code> 's own internal computation, and by the time <code>calc_mapq</code> runs it has already mutated $J2$ in place, so in practice this tag <i>does</i> show the clamped value whenever a MAPQ was computed with no real second-best; only relevant if reimplementing without that in-place mutation).
Jdiff:f:	float	$J - J2$.
sh:f:	float	The seed-heuristic value (h_{seed} , §2.6) at the moment this bucket was accepted.
strand:i:	int	<code>same_strand_seeds</code> signed tally (§4.5).
t:f:	float	Wall-clock seconds spent mapping this read (never reproducible run-to-run; exclude when diffing output against a golden/reference file).
b:s:	string	Best mapping's bucket location, formatted (<code>segm_id,b</code>).
b2:s:	string	Second-best's bucket location, or the sentinel $(-1, -1)$ (§4.6) if none.

13.2.2 Run-wide/per-read summary tags (GlobalMappingStats)

Tag	Type	Meaning
<code>k:i:</code>	int	k -mer length used (CLI <code>-k</code>).
<code>gt_J:f:</code> , <code>gt_C:f:</code> , <code>gt_C_bucket:f:</code>	float	Ground-truth Jaccard/containment fields — always <code>-1.0</code> ; declared and printed but never actually computed anywhere in the reference source (the real ground-truth comparison output is the entirely separate <code>-v 2</code> tag block, §13.3, not these three). Kept as permanent <code>-1.0</code> placeholders for output-format fidelity.
<code>seeds:i:</code>	int	Seed budget S actually used for this read (§7.2).
<code>max_seed_matches:i:</code>	int	Largest <code>hits_int</code> among any seed consumed this read.
<code>seed_matches:i:</code>	int	Total reference hits examined while matching seeds this read.
<code>total_matches:i:</code>	int	Same as <code>seed_matches:i:</code> at present (both are incremented together in <code>match_seeds</code> ; kept as two separately-named counters for forward compatibility with a scheme that might one day count additional non-seed matches separately).
<code>match_inefficiency:f:</code>	float	<code>total_matches / intersection</code> — how many candidate-match examinations were needed per k -mer that actually ended up in the reported mapping; a rough proxy for how much the seed heuristic saved (lower is better; the reference tool's own runtime warning system, §13.5, flags a run where this ratio implies the heuristic isn't earning its keep).
<code>seeded_buckets:i:</code>	int	Distinct buckets touched by <code>match_seeds</code> this read.
<code>final_buckets:i:</code>	int	Buckets that survived pruning <i>and</i> beat the running threshold at least once during <code>match_rest</code> (counted across <i>both</i> the best- and second-best search passes).
<code>FFTP:f:</code>	float	Always <code>-1.0</code> — a false-discovery-rate calculation that exists in the source as a fully commented-out, never-invoked function (<code>calc_FDR</code>); kept as a permanent placeholder for the same reason as <code>gt_J/gt_C</code> above.

13.3 Ground-truth diagnostic tags (`-v 2` only)

When run with `-v 2` against *simulated* reads whose FASTA header encodes the true origin (format: *anything!segment_name!start! end!strand*, e.g. `S1_21!NC_060948.1!57693539!57715501!+` — exactly 5 `!`-separated fields), an additional block of tags is appended, and (if an output prefix was configured) a parallel, more detailed TSV file (`<prefix>.paul.tsv`) is written, one row per read, both mapped and unmapped.

Tag	Type	Meaning
<code>gt_segms:</code>	string	True segment name (parsed from the read name).
<code>gt_start_nucl:i:</code> , <code>gt_end_nucl:i:</code>	int	True start/end nucleotide positions (parsed from the read name).
<code>bucket_l:i:</code>	int	This read’s bucket half-length ℓ .
<code>gt_b_l:s:</code> , <code>gt_b_r:s:</code> , <code>gt_b_next:s:</code>	string	The three buckets straddling the true position: one half-length before, at, and one half-length after $\lfloor start/\ell \rfloor$.
<code>gt_M_l:i:</code> , <code>gt_M_r:i:</code> , <code>gt_M_next:i:</code>	int	Number of (seed, hit) matches collected in each of those three buckets.
<code>gt_J_l:f:</code> , <code>gt_J_r:f:</code> , <code>gt_J_next:f:</code>	float	Best included-Jaccard score (§10.6) achievable in each of those buckets, restricted to windows near the true span’s length.
<code>gt_C_l:f:</code> , <code>gt_C_r:f:</code> , <code>gt_C_next:f:</code>	float	Best containment score in each, using the (dead-parameter, see Chapter 16) ground-truth <code>bestFixedLength</code> .
<code>gt_C_l_lmax:f:</code>	float	Numerically identical to <code>gt_C_l:f:</code> — both call sites pass different values for a parameter the callee never reads (Chapter 16).
<code>gt_C_r_lmax:f:</code>	float	Always <code>-1.0</code> : declared, printed, but never assigned in the constructor at all (a distinct defect from the <code>gt_C_l_lmax</code> redundancy above — this one is a genuinely missing assignment, not a no-op parameter).

The parallel `.paul.tsv` file additionally reports, per read: the count of buckets whose included-Jaccard / containment score clears θ at all (`#J>theta`, `#C>theta`), a bracketed listing of exactly which buckets those are with their individual scores (`J>theta`, `C>theta`), the maximum score seen across all buckets (`maxJ`, `maxC`), and the read’s own raw sequence (`P`, the final column) — this file is intended for offline analysis of pruning/scoring quality against known-correct answers, not for routine use.

13.4 Unmapped-read record

Per the fix recorded in Chapter 16/11, an unmapped read is written as a minimal, well-defined 12-column PAF-shaped line (broadly following the samtools/minimap2 convention for “no alignment”):

```
query_id read_len 0 0 * * 0 0 0 0 0 0
```

i.e., query name and length populated, strand and target name as `*` (unknown), every other numeric field zero. No diagnostic tags are appended for unmapped reads (there is no candidate mapping to describe). This record is only ever written to the `.unmapped.paf` file described above, which itself is only created when an output prefix (CLI `-z`) is given.

13.5 Run-wide summary printed to stderr

At the end of a run, three human-readable reports are printed to standard error (never standard output, so they never interleave with PAF data): (1) per-read averages/percentages for kmer/match/bucket counts, mirroring the `GlobalMappingStats` tag definitions above but aggregated across the whole run; (2) a timing breakdown (seconds and percentage-of-total spent in each pipeline stage, plus a “range ratio” = slowest read / fastest read for that stage, useful

for spotting pathological reads); and (3) a short list of heuristic warnings (highlighted in color) triggered when: fewer than 95% of reads mapped; `match_inefficiency` implies the seed heuristic isn't paying for itself (average possible-matches-to-total-matches ratio below $10\times$); more reads got `mapq=60` than `mapq=0` is *violated* (i.e. a warning fires when low-confidence mappings outnumber high-confidence ones); or any single pipeline stage (seed matching, refinement, or specifically the second-best search within refinement) consumes an outsized share of total runtime. These are operational/performance diagnostics, not part of the mapping algorithm proper, and are safe to omit or reshape freely in a from-scratch implementation.

Chapter 14

Command-Line Parameter Reference

Flag	Name	Default	Validation	Effect
-p	pattern (reads file)	— (required)	must exist	Path to the query reads, FASTA/FASTQ, optionally gzip-compressed.
-s	text (reference file)	— (required)	must exist	Path to the reference, FASTA, optionally compressed.
-k	ksize	15	> 0	k -mer length, Chapter 5.
-r	hashratio	0.05	(0, 1]	FracMinHash keep-fraction r , §2.3.3.
-S	max_seeds	unset	> 0 if given	Dead/inert: parsed, validated, and printed in the parameter dump, but never read by any algorithmic code path (confirmed by exhaustive cross-reference). Kept for CLI compatibility only.
-M	max_matches	unset (no cap)	> 0 if given	Caps reference occurrences per k -mer hash before indexing, §6.2.3.
-t	threshold	0.9	[0, 1]	Acceptance threshold θ , used throughout seeding (§7.2) and refinement (§10.5).
-d	min_diff	0.02	≥ 0	<i>min_diff</i> : MAPQ confidence cutoff (§12), second-best search threshold factor (§10.5), and an input to the seed-budget formula (§7.2) — one parameter, three distinct uses.
-o	max_overlap	0.5	[0, 1]	Overlap fraction above which a second-best candidate is rejected as a restatement of the best, §10.5.1.
-m	metric	Containment	one of {Containment, Jaccard, bucket_SH, bucket_LCS}	Selects the refinement scoring function, §3.3, Chapter 10.
-z	params (output prefix)	unset	—	If set: (1) parameters are written to <prefix> as TSV instead of a human-readable dump to stderr; (2) unmapped reads are written to <prefix>.unmapped.paf (§13.4); (3) if -v 2 is also set, ground-truth rows are written to <prefix>.paul.tsv (§13.3).

Flag	Name	Default	Validation	Effect
-v	verbose	0	{0, 1, 2}	0: silent. 1: reserved (no distinct behavior beyond 0 found). 2: enables the ground-truth diagnostic path, §13.3 — requires reads whose FASTA headers are ground-truth-encoded; using -v 2 on ordinary reads is a usage error (the ground-truth parser will reject the header format; a from-scratch implementation should surface this as a clear error rather than the reference’s own undefined/crashing behavior on malformed input in this mode).
-n	normalize	false	—	Dead/inert: same status as -S — parsed, printed, never read.
-P	no_bucket_pruning	false	—	Disables seed-heuristic pruning entirely, §9.3.
-B	one_sweep	false	—	In the reference source: parsed and templated but <i>never branched on anywhere in the mapper</i> — fully dead despite having a descriptive help string (“disregards seed heuristic and runs one sweep on all matches”). This document’s companion re-implementation gives it a concrete, best-effort meaning (consume every seed rather than the budget-limited <i>S</i> , §7.2) since it was asked to make all configuration combinations real; see Chapter 16 for the full discussion and why this is explicitly a judgment call rather than a recovered original behavior.
-F	abs_pos	false	—	Selects nucleotide-position vs. sketch-index-position coordinate space for bucket/window arithmetic throughout Chapters 8–10.
-a	sam	false	—	Dead/inert , and additionally the entire SAM/CIGAR output code path this flag would have selected is itself fully commented out in the reference source (it depends on an edit-distance library that is otherwise unused). A from-scratch implementation targeting only the mapping functionality this document describes should not implement this flag at all.

Table 14.1: Every shmap CLI parameter, its validation rule, and its actual (not merely documented) effect.

14.1 Parameters not exposed on the CLI but referenced throughout this document

Symbol	Derived from	Defined in
θ_2	$\theta - min_diff$	§7.2, §11.2
S (seed budget)	θ_2 , read sketch size m	§7.2
ℓ (bucket half-length)	read sketch size m (or read length, if <code>abs_pos</code>)	§8.1.1
$\ell_{\min}$	hardcoded constant, 5	§8.1.1

Chapter 15

Complexity Analysis

Notation: n = total reference length (nucleotides); r = FracMinHash keep-fraction (`-r`); $|P|$ = one read's length; $m = |\hat{P}| \approx r|P|$ = its sketch size; $S \leq m$ = seed budget actually consumed (§7.2); d = a consumed seed's reference-hit count (capped by `-M` when set, §6.2.3); u = number of distinct buckets touched by one read; N = number of reads.

15.1 Indexing (once per run)

Step	Time	Space
Sketch every segment (Ch. 5)	$O(n)$	$O(rn)$ (the kept k -mers)
Populate <code>h2single/h2multi</code> (Ch. 6)	$O(rn)$ amortized	$O(rn)$
Sort each multi-hit list (Ch. 6)	$O(\sum_h d_h \log d_h) \leq O(rn \log d_{\max})$	in place

Total: $O(n + rn \log d_{\max})$ time, $O(rn)$ space, independent of the number of reads that will subsequently be mapped.

15.2 Per read

Step	Time	Notes
Sketch the read (Ch. 5)	$O(P)$	
Group into seeds (Ch. 7)	$O(m \log m)$	two sorts of size m
Match seeds into buckets (Ch. 8)	$O(S \cdot \bar{d})$	$\bar{d} =$ avg. hit count of a con- sumed seed, $\leq$ -M cap
Sort/dedup touched buckets (Ch. 8)	$O(u \log u)$	$u \ll$ n/ℓ in prac- tice
Seed-heuristic pruning (Ch. 9)	$O(u)$ amortized, typically $O(1)$ per pruned bucket	see dis- cus- sion be- low
Refine surviving buckets (Ch. 10)	$O(\ell)$ per surviving bucket	$\ell =$ bucket half- length $\approx$ m
MAPQ (Ch. 12)	$O(1)$	
Emit PAF line (Ch. 13)	$O(1)$	

Why pruning is (empirically) sublinear in the number of candidate buckets. The seed-heuristic bound (§2.6) only needs a bucket’s running (`seeds`, `matches`) counters, and tightens monotonically as more seeds are examined. In the common case where a repetitive genome generates many candidate buckets but only one (or a handful) is the true origin, every wrong bucket accumulates “known misses” quickly (most consumed seeds simply do not have a reference hit inside a wrong bucket’s span), so its bound drops below θ after only a small, effectively $O(1)$ -in-practice number of extension steps — this is an empirical/probabilistic claim (it depends on the genome’s repeat structure and θ), not a worst-case guarantee: an adversarial input (e.g. a reference consisting almost entirely of one repeated motif) can force every bucket to survive many extension rounds before being resolved, degrading toward the $O(u \cdot m)$ worst case

of exhaustively extending every bucket with every seed.

Total per-read cost. Combining the above, and noting u is typically small relative to n/ℓ (only a minority of possible bucket positions are ever touched by any real read’s seeds), a read’s total mapping cost is, in the typical case, dominated by $O(m \log m)$ (seeding) plus $O(\ell)$ (refining the eventual winner and perhaps a few near-misses) — i.e. **roughly independent of total reference size** n once the index is built, which is exactly the scaling property that makes sketch-based mapping practical at genome scale. Over N reads: $O(n + rn \log d_{\max} + N(m \log m + \ell))$ total.

15.3 Memory

Peak memory is dominated by the index: $O(rn)$ for the stored per-segment sketches (`RefSegment.kmers`) plus $O(rn)$ for the hash maps (`h2single/h2multi`, one entry per kept k -mer occurrence, with `h2multi` paying a small constant-factor overhead per entry for its vector-of-hits indirection). Per-read working memory (seed table, `diff_hist`, touched-bucket list) is $O(m + u)$ and is released (or reused, in the reference/port’s actual implementation, which clears rather than reallocates, §8.2.1) between reads, so it does not accumulate across a run.

15.4 Practical scaling knobs

Parameter	Increasing it...	... trades against
-r (keep fraction)	more sensitivity (finds weaker true matches)	linearly more index memory, more seeds per read, more re-finement work per bucket
-k (k -mer length)	fewer spurious matches (larger alphabet space per k -mer)	more read-s/regions missed entirely if error rate $\times k$ approaches 1
-t (threshold θ)	fewer accepted mappings, smaller seed budget S	lower recall in genuinely divergent regions
-M (max_matches cap)	fewer index entries scanned per common k -mer	silently drops legitimate signal from extremely repetitive true loci once capped

Chapter 16

Catalog of Defects and Dead Code in the Reference Implementation

Every entry below was found during an exhaustive, function-by-function cross-reference of the reference C++ source against an independent re-implementation, and confirmed either by constructing a triggering test case or by an exhaustive grep-based call-site audit (for dead-code claims: a claim of “zero call sites” means every reference to the symbol in the entire source tree was individually inspected). None of these were assumed from documentation or comments alone.

16.1 Summary table

#	Defect	Affects live output?	Severity	Recommended handling
D1	Uninitialized LUT arrays	Only non-ACGT input	Latent UB	Zero-initialize (§16.2)
D2	<code>kmers_notmatched</code> miscount	Diagnostic only	Cosmetic	Reorder the increment (§16.3)
D3	<code>more_seeds_if_cheap</code> unreachable	No (dead)	None	Optional to wire in (§16.4)
D4	Hardcoded 100-segment cap	Crashes on > 100 contigs	Usability	Size dynamically (§16.5)
D5	<code>Buckets::size()</code> dead stub	No (dead)	None	Drop (§16.6)
D6	<code>Mapping::do_overlap</code> dead	No (dead)	None	Drop (§16.6)
D7	<code>Matcher::do_overlap/lost_correct_mapping</code> dead	No (dead)	None	Drop (§16.6)
D8	Dead <code>lmax</code> parameter in ground-truth <code>bestFixedLength</code>	Diagnostic only	Cosmetic/confusing	Drop the parameter (§16.7)
D9	Off-by-one in <code>bestIncludedJaccard</code>	Diagnostic only (-v 2)	Real correctness bug	Fix indexing (§16.8)
D10	Missing bounds clamp in <code>collect_matches</code>	Diagnostic only, potential crash/OOB	Latent bug	Clamp to segment length (§16.9)
D11	<code>RefSegment::seq</code> unused waste	No (memory only)	Efficiency	Omit field (§16.10)
D12	Dead <code>-a</code> /SAM output path	No (dead)	None	Omit entirely (§16.11)
D13	Dead <code>-S/-n</code> CLI flags	No (dead)	None	Keep parsed-but-inert for compatibility (§16.12)
D14	<code>one_sweep/-B</code> never branched on	No (dead)	Usability/API honesty	Implement a concrete meaning (§16.13)
D15	Per-read counters never reset	Yes — live PAF tags	Real correctness bug	Fully reset every read (§16.14)
D16	Unconditional dereference of empty optional	Yes — crashes/UB on every unmapped read	Real correctness bug (UB)	Emit a defined minimal record (§16.15)
D17	<code>gt.C_r.lmax</code> never assigned	Diagnostic only (-v 2)	Cosmetic	Leave unassigned or compute it properly (§16.16)
D18	PAF columns 10/11 always repeat query length	Yes, if spec-conformance matters	Cosmetic/compliance	Populate real values if spec conformance is required (§16.17)
D19	<code>Timers::secs</code> 's try/catch never catches anything	No (silent no-op)	Latent	Handle missing keys directly (§16.18)
D20	<code>ParsedQueryId::parse</code>	No	Style only	Simplify (§16.19)

16.2 D1: Uninitialized hash lookup tables

Location: `FracMinHash` construction, `src/sketch.h` (§5.2). **Description:** `LUT_fw/LUT_rc` are fixed 256-entry arrays; only the 8 entries for `ACGTacgt` are ever written. Any other input byte (most commonly `N`) reads uninitialized memory when used as a hash contribution — undefined behavior in C++, and not even a fixed garbage value (it depends on prior stack contents at the time `sketch()` first runs). **Trigger:** any reference or read sequence containing a non-ACGT byte. **Fix:** zero-initialize both tables before filling in the 8 real entries. This is a strict safety improvement with zero behavioral change on ACGT-only input (the overwhelming majority of real data), and was adopted without further discussion in the companion re-implementation.

16.3 D2: `kmers_notmatched` always reports the full sketch size

Location: `unique_elements_with_info`, `src/shmap.h` (§7.1). **Description:** the increment `if (hits_in_t > 0) nonzero += strike;` is placed textually after `strike` has already been reset to 0 for the next group, so it always adds zero; `kmers_notmatched` (a diagnostic counter only) therefore always reports the entire sketch size regardless of actual match rate. **Impact:** cosmetic only — feeds one end-of-run percentage in the `stderr` summary (§13.5), nothing else. **Fix:** move the increment before the reset (or reorder the two statements).

16.4 D3: `more_seeds_if_cheap` is fully implemented but unreachable

Location: `src/shmap.h` (§7.2). **Description:** a complete, presumably-tested function to opportunistically extend the seed budget when the next seeds are (almost) unique in the reference; its only call site in `map_read` is commented out. **Impact:** none on shipped behavior (never runs). **Recommendation:** a from-scratch implementation may wire it in (a plausible, low-risk recall improvement) or omit it to match shipped behavior exactly; either is defensible.

16.5 D4: Hardcoded 100-segment cap

Location: `Buckets` construction, `src/buckets.h` (§8.2). **Description:** `MAX_SEGMENTS = 100;` construction throws for references with more segments. **Trigger:** draft/scaffolded assemblies with hundreds or thousands of small contigs — not a rare input in practice. **Fix:** size the per-segment bucket storage to the reference’s actual segment count.

16.6 D5–D7: Dead stubs and unreachable methods

Three separate pieces of code are fully defined but have **zero** call sites anywhere in the source tree (verified exhaustively, not just “no obvious caller”):

- `Buckets::size()` (`src/buckets.h`) — literally `return -1;` with a `// TODO: implement comment`.
- `Mapping::do_overlap` (`src/sketch.h`) — a boolean overlap check, superseded everywhere by the (used) `Mapping::overlap` that returns a fraction (§10.5.1).
- `Matcher::do_overlap` and `Matcher::lost_correct_mapping` (`src/refine.h`) — ground-truth-overlap-checking helpers, superseded by the `-v 2` diagnostic path (§13.3) actually used.

Recommendation: omit all three from a from-scratch implementation; they contribute no behavior.

16.7 D8: Dead `lmax` parameter in the ground-truth `bestFixedLength`

Location: `src/refine.h`, the `Matcher`-scoped `bestFixedLength` (§10.6). **Description:** the function signature accepts an `lmax` parameter, and every call site dutifully computes and passes a value for it (`2*bucket_l` or `bucket_l`), but the function body’s actual window-width check uses `P_sz` instead — a commented-out `lmax`-based condition sits directly above the live one, evidence the author switched approaches and left the now-ornamental parameter (and its call-site computation) in place. **Consequence:** the two ground-truth tags `gt_C.l.lmax` and `gt_C.l` (§13.3), whose call sites differ only in what they pass for this dead parameter, are numerically *identical* in practice. **Fix:** drop the parameter; it does nothing.

16.8 D9: Off-by-one in `bestIncludedJaccard` (real bug)

Location: `src/refine.h`, `Matcher::bestIncludedJaccard` (§10.6). Full technical description already given in §10.6; repeated here for completeness of this catalog. **Description:** the function’s window-growing loop scores mid-iteration using the position *one before* the just-processed element (`prev(r)`) for the reported span, while `intersection` at that point already reflects the just-processed element — an internal inconsistency that can and does produce `intersection > span`, violating the metric’s own $[0, 1]$ -boundedness invariant. The original source carries an explicit `// TODO: should it be prev(r) instead?` comment on this exact line. **Trigger:** confirmed via a small constructed example (a ~ 50 bp read against a ~ 200 bp synthetic reference, exercising the `-v 2` ground-truth “included Jaccard” computation). **Impact:** affects only the `-v 2` diagnostic tags/TSV, never the primary PAF output or any mapping decision (this sweep is not used by the main scoring path, §10.2, which has the analogous computation *and gets it right* by scoring only after its inner loop fully exits). **Fix:** score using the *current*, just-processed right-boundary element consistently (matching what `intersection` already counts), not the one before it.

16.9 D10: Missing bounds clamp in ground-truth `collect_matches`

Location: `src/refine.h`, `Matcher::collect_matches`. **Description:** this function computes a bucket’s `[start, end)` range into a segment’s sketch array but does not clamp `end` to the segment’s actual length — for the last bucket of a segment, a bucket’s nominal end can run past the sketch array’s bounds, risking an out-of-bounds read. The same file’s own (separately dead, D7) `do_overlap` function *does* apply exactly this clamp for the same quantity, suggesting the omission here is an oversight rather than intentional. **Fix:** clamp `end` to `min(end, len(segment.kmers))` before iterating, mirroring the clamp already present elsewhere in the same file.

16.10 D11: `RefSegment::seq` is stored but never read (in the shipped feature set)

Location: `src/sketch.h` (§4.8). **Description:** the full raw nucleotide sequence of every reference segment is retained in memory, but the only code that ever reads it (a SAM-output/edit-distance feature) is entirely commented out and unreachable. For large references this is a substantial, unnecessary memory cost (roughly doubling index memory beyond the sketch itself). **Recommendation:** omit the field entirely in a from-scratch implementation targeting the mapping functionality this document describes; only retain it if also implementing base-level alignment output.

16.11 D12: Dead `-a`/SAM output path

Location: CLI parsing (`src/io.h`) and `src/sketch.h`. **Description:** the `-a` flag (`sam`) is parsed and stored but never read by any live code; the SAM-format output function it would have selected is fully commented out (and itself depends on an edit-distance library, `edlib`, that is otherwise entirely unused by the shipped binary). **Recommendation:** omit this flag from a from-scratch implementation rather than reproduce a no-op switch.

16.12 D13: Dead `-s/-n` CLI flags

Location: CLI parsing (`src/io.h`). **Description:** `max_seeds` (`-S`) and `normalize` (`-n`) are parsed, range-validated, and included in the parameter dump, but neither is ever read by any algorithmic code. **Distinction from D12:** unlike `-a`, these are not accompanied by a larger commented-out feature — they appear to be either forward-looking placeholders or remnants of a removed feature. **Recommendation:** unlike the confirmed-and-explicitly-flagged `-a` case, a from-scratch implementation should by default *keep* these as accepted-but-inert CLI parameters (for command-line compatibility with existing scripts/pipelines that may already pass them) rather than unilaterally removing CLI-visible surface area that wasn't specifically identified for removal.

16.13 D14: `one_sweep/-B` has no implemented behavior

Location: template parameter and CLI flag threaded throughout `src/shmap.h`, `src/mapper.cpp`. **Description:** unlike `no_bucket_pruning/-P` (which does gate real behavior, §9.3) or `abs_pos/-F` (which gates coordinate-space arithmetic throughout Chapters 8–10), `one_sweep` is threaded through every relevant function signature and even given a descriptive CLI help string (“disregards seed heuristic and runs one sweep on all matches”), but is never once tested inside any conditional in the mapper’s actual logic — confirmed via exhaustive grep. **Recommendation:** since this document’s companion implementation was tasked with making every configuration combination genuinely meaningful (rather than silently inert), it defines a concrete best-effort semantics matching the flag’s own help text: consume *every* seed (`S := len(seeds)`) rather than the theta-derived budget (§7.2), leaving the rest of the pipeline (bucket pruning, refinement) unchanged. This is an explicit design choice filling a gap the reference source left open, not a recovered original behavior — a from-scratch implementation is free to define `one_sweep` differently (e.g. literally skip bucket-based clustering and score one global sweep over every raw match, which is closer to a literal reading of the help text but a substantially larger undertaking) as long as it documents which interpretation it chose.

16.14 D15: Per-read counters are never actually reset (real bug, affects live output)

Location: `map_read`, `src/shmap.h` (§11.2.1). **Description:** the call that would fully reset the per-read `Counters` object (`C.clear()`) is commented out; the subsequent re-initialization only names a subset of the counter keys the function goes on to increment. The following counters are left unreset across every read a single mapper instance processes: `read_len`, `total_matches`, `possible_matches`, `kmers_sketched`, `kmers_unique`, `kmers_seeds`, `seeded_buckets`. Because the (also unreset) running counters object is merged into the run-wide totals *after every single read*, these specific counters compound super-linearly across a run. **Concretely affected live output:** `total_matches:i:` and its derived `match_inefficiency:f:` PAF tag (§13) are read *directly* from this same, never-reset counters object *before* the end-of-read merge into

run-wide totals — meaning every mapped read’s own reported `total_matches:i:` value already reflects leftover accumulation from every previous read the same mapper instance processed, not just the current read. This is the most consequential defect in this catalog: it corrupts an observable, per-mapping output field, not merely an end-of-run summary statistic. **Confirmed** by observing, in a controlled two-read test, `total_matches:i:` values that were consistent per-read only after applying the fix below; without it, the second read’s value visibly incorporated the first read’s contribution. **Fix:** fully clear the per-read counters object at the start of every read, and pre-register (at zero) every counter name the function’s call graph touches (the complete, corrected list is: every name in the reference source’s own partial list, *plus* the seven listed above).

16.15 D16: Unconditional dereference of an empty optional on unmapped reads (real bug, UB)

Location: `map_read`, `src/shmap.h` (§11.2.2). **Description:** `best->set_global_stats(...)` and `best->print_paf(...)` are called unconditionally, textually after the point where the code has already branched on whether `best` holds a value or not — for every unmapped read, this dereferences an empty `std::optional<Mapping>`, which is undefined behavior in C++ (it does not reliably crash; depending on standard-library internals it may silently read whatever a default-constructed- looking object’s memory layout happens to contain). **Trigger:** any read that does not map (a normal, expected, frequently-occurring outcome — the surrounding code clearly anticipates it, since a dedicated `unmapped_out` stream and `lost_on_seeding/lost_on_pruning` counters all exist specifically to track this case). **Fix:** guard both calls behind the same `if (best)` check already present a few lines above, and define an explicit, well-formed output for the unmapped case instead — this document’s companion implementation writes a minimal 12-column PAF-shaped record (§13.4) rather than silently omitting unmapped reads from output entirely, on the reasoning that the reference source’s own `unmapped_out` stream was clearly intended to receive *some* record per unmapped read, even though the bug above meant it never cleanly did.

16.16 D17: `gt_C.r_lmax` is declared, printed, but never assigned

Location: `AnalyseSimulatedReads` constructor, `src/analyse_simulated.h` (§13.3). **Description:** distinct from D8 (which is a dead *parameter*) — this is a genuinely missing assignment. The constructor computes and stores `gt_C.l_lmax` but has no corresponding `gt_C.r_lmax = ...` line at all, despite both fields being declared and both being unconditionally printed in *both* the `-v 2` PAF tags and the `.paul.tsv` output. **Impact:** the `gt_C.r_lmax:f:` tag / TSV column always shows the default `Mapping`’s sentinel score, `-1.000`, for every read, in every run. **Recommendation:** since this diagnostic path has zero automated test coverage to indicate the intended value, a from-scratch implementation may either leave the field unassigned (matching shipped behavior exactly) or assign it the obviously-analogous computation (mirroring `gt_C.l_lmax` but for the right-side ground-truth bucket) — both are defensible; this document’s companion implementation chose to leave it unassigned, to avoid inventing behavior for a code path with no oracle to validate against.

16.17 D18: PAF columns 10/11 always repeat the query-length column

Already described in full in §13; included here for completeness of this catalog. Marked with the reference source’s own `// TODO: fix` comment.

16.18 D19: `Timers::secs`'s exception handling can never actually fire

Location: `src/utils.h`. **Description:** `Timers::secs(name)` wraps `timers_.at(name).secs()` in a `try { ... } catch (const std::runtime_error&)` intended to gracefully handle a missing timer name. However, `std::unordered_map::at` throws `std::out_of_range` on a missing key, which does *not* derive from `std::runtime_error` in the C++ standard exception hierarchy (both derive independently from `std::exception`) — so this catch clause can never actually trigger for the failure mode it was written for. In practice this is masked because every timer name actually queried has always been previously `.start()`'d (which auto-vivifies the entry), so the missing-key path is never exercised by the shipped tool. **Recommendation:** a from-scratch implementation should simply handle the “timer not found” case directly (e.g. return 0.0 and log, which was evidently the original intent) rather than reproduce a try/catch that provides no actual safety net.

16.19 D20: `ParsedQueryId::parse` catches a parse error only to rethrow it

Location: `src/utils.h`. **Description:** when a ground-truth-encoded read name's numeric fields (start/end position) fail to parse as integers, the function catches the resulting exception, logs an error message, sets a (about-to-be-discarded) validity flag to false, and then rethrows the same exception unconditionally — so the net behavior is identical to not catching it at all, just with an extra log line first. **Distinction from the two graceful cases the same function handles** (no `!`-delimiter present at all; wrong number of `!`-delimited fields), both of which *do* return a clean “not a ground-truth name” result: a malformed *numeric* field within an otherwise correctly-shaped name is treated as a hard failure, not a graceful “not ground-truth-encoded” result. **Recommendation:** this is reasonable, defensible behavior for a diagnostic feature that assumes well-formed simulated-read names (only used in the opt-in `-v 2` mode); a from-scratch implementation should simply let the parse error propagate (panic/exception) without the pointless catch-log-throw wrapper, since the wrapper adds a log message but changes nothing else.

Appendix A

Notation, Glossary, and Quick Index

A.1 Notation summary

Symbol	Meaning	Defined
k	k -mer length	§5.2, CLI <code>-k</code>
r	FracMinHash keep-fraction	§2.3.3, CLI <code>-r</code>
$H_{\max}$	Maximum hash value ($2^{64} - 1$)	§5.4
n	Total reference length	§15
$ P $	Read length (nucleotides)	
m	Read's sketch cardinality	§7.1
S	Seed budget (occurrences) consumed	§7.2
θ	Acceptance threshold	CLI <code>-t</code>
θ_2	$\theta - \text{min_diff}$	§7.2
min_diff	MAPQ/threshold margin	CLI <code>-d</code>
ℓ	Bucket half-length	§8.1.1
$\ell_{\min}$	Minimum allowed ℓ ($= 5$)	§8.1.1
h_{seed}	Seed-heuristic bound	§2.6
$J(A, B)$	Jaccard similarity	§2.3.2
$C(A, B)$	Containment index	§2.3.3
f_1, f_2	Best / second-best mapping score	§12

A.2 Glossary

Bucket

A fixed-geometry, overlapping candidate window along one reference segment (Chapter 8).

Canonical k -mer

A k -mer identified by a strand-agnostic combination of its forward and reverse-complement hash (§2.1.1).

Containment index

Fraction of one set's elements found in another; shmap's primary scoring metric (§2.3.3).

FracMinHash

A hash-threshold-based sketching scheme giving length-proportional sketch size (§2.3.3).

Hit A k -mer's occurrence in the *reference* (§4.3).

Jaccard similarity

Intersection over union of two sets (§2.3.2).

MAPQ

Mapping quality, a Phred-like confidence score for a reported mapping (§2.7, Chapter 12).

Minimizer

The smallest-hash k -mer in a sliding window; an alternative sketching scheme shmap does *not* use (§2.3.1).

PAF Pairwise mApping Format, shmap’s output format (Chapter 13).

Rolling hash

An $O(1)$ -amortized-update hash recurrence over sliding windows (§2.2).

Seed One distinct k -mer hash from a read’s sketch, with reference/read occurrence metadata attached (§4.4).

Seed heuristic

The closed-form pruning upper bound central to shmap’s performance (§2.6, Chapter 9).

Sketch

A subsampled representative subset of a sequence’s k -mers (§2.3.2–§2.3.3).

A.3 Quick algorithm index

Function	Section	Purpose
<code>sketch</code>	§5	FracMinHash rolling hash
<code>build_index</code>	§6	Reference k -mer index
<code>unique_elements_with_info</code>	§7.1	Group read k -mers into seeds
<code>match_seeds</code>	§8.4	Populate buckets from the seed budget
<code>hseed</code>	§9	Seed-heuristic bound formula
<code>seed_heuristic_pass</code>	§9.3	Incremental pruning loop
<code>bestFixedLength</code> (SHMapper)	§10.2	Main Containment/Jaccard sweep
<code>lcs</code>	§10.4	bucket_LCS metric
<code>findBestMapping</code>	§10.3	Metric dispatch
<code>match_rest</code>	§10.5	Best/second-best search
<code>calc_mapq</code>	§12.3	MAPQ formula
<code>map_read</code>	§11.2	Full per-read driver

A.4 Minimal reference implementation checklist

A from-scratch implementation is functionally complete once it has:

1. Bit-exact FracMinHash sketching (Chapter 5), verified via the reverse-complement symmetry test (end of §5).
2. A reference index with correct single/multi occurrence splitting and sorted multi-hit lists (Chapter 6).
3. Rarest-first seed grouping with correctly-collected `pmatches` (Chapter 7).
4. Overlapping bucket geometry with the 50% overlap rule applied consistently (Chapter 8).
5. The seed-heuristic bound implemented and unit-tested against hand-derived examples (Chapter 9).
6. All four scoring metrics (Chapter 10), each independently verified (e.g. against a brute-force $O(n)$ containment/Jaccard computation on small synthetic inputs).

7. Best/second-best search with overlap exclusion and MAPQ (Chapters 11– 12).
8. Well-defined (not undefined-behavior) handling of the unmapped case (§13.4).
9. PAF output matching §13’s column/tag definitions.

Chapter 16 should be read in full before finalizing any of the above, since several entries there materially change what “correct” means for a specific function relative to a naive reading of the reference source alone.